

U

O

W

CSIT305 Emerging Information Technologies and their Applications



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Spatial Data and Computing



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Logistics

- (1) Advanced Database Overview
- (2) Spatial Data and Computing
- (3) Graph Data and Computing
- (4) Text Data and Processing

Multi-Modal Database



- (1) AI4SE: artificial intelligence for software engineering
- (2) Software Engineering in Cloud Computing I
- (3) Software Engineering in Cloud Computing II

Software



- (1) IoT and Edge Computing
- (2) Digital Twin Technology
- (3) Blockchains and Smart Contracts
- (4) Emerging Cyber-Attacks

Networking

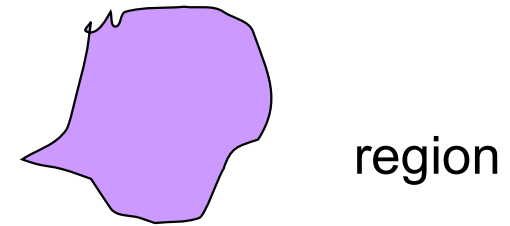


Outline

- Spatial Data
 - Spatial Query Types
 - Spatial Index Structure
 - Spatial Query Processing
- Spatial Textual Data
 - Spatial Textual Index Structure
 - Spatial Textual Query Processing
- Applications

Spatial Data

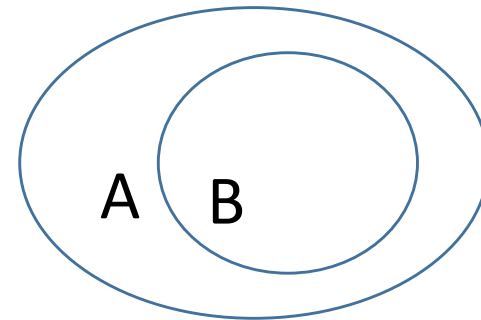
- Spatial Data Types



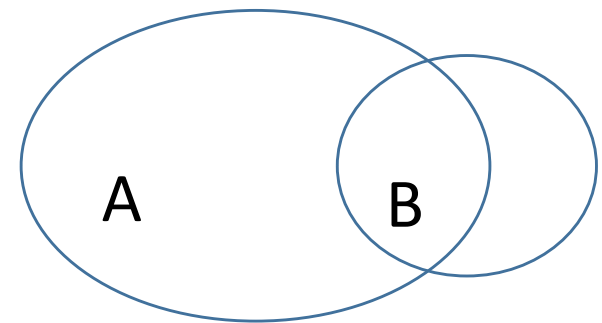
- Point : 2 real numbers
- Line : sequence of points
- Region : area included inside n-points

Spatial Data

- Spatial Relationships
 - Topological relationships:
 - adjacent, contains, intersects, disjoint etc.
 - Direction relationships:
 - Above, below, north_of, etc.
 - Metric relationships:
 - “distance < 100 m”



A contains B



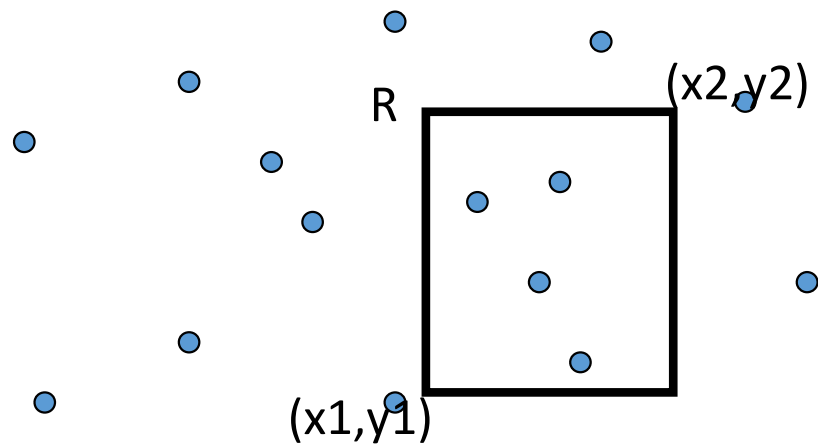
A intersects B

Spatial Query Types

- Window queries
- Range queries
- K nearest neighbor queries

Window Queries

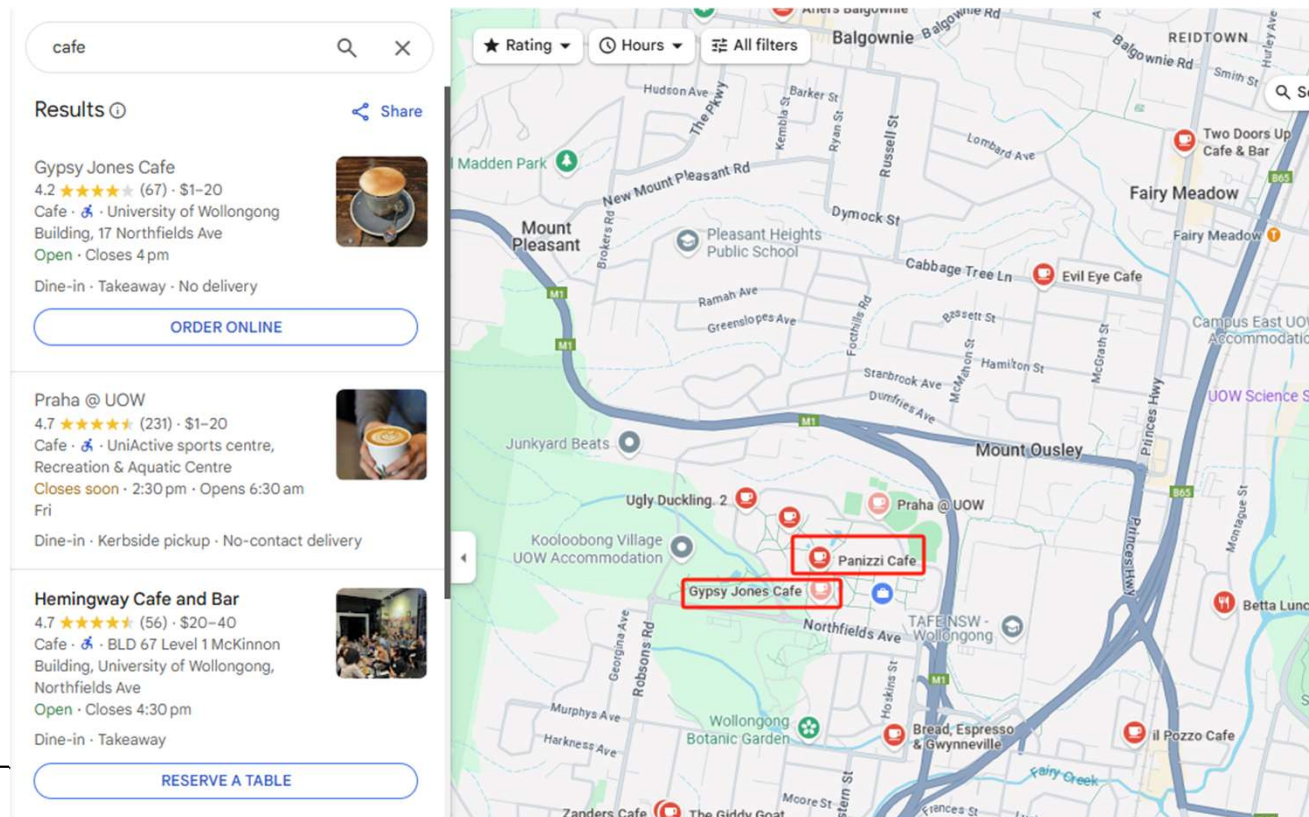
- Given n points and a query rectangle, find all the points enclosed in the rectangle



```
if (x1 <= o.x && x2 >= o.x && y1 <= o.y && y2 >= o.y )
```

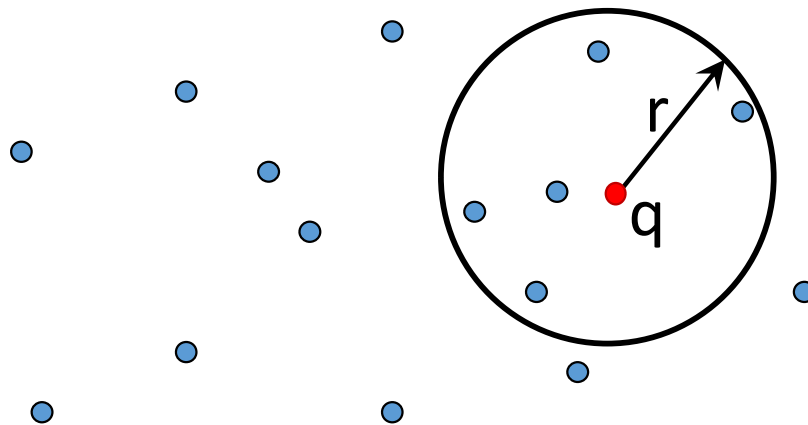

Window Queries

- Example: find all the cafes in Wollongong CBD



Range Queries

- Given n points, a query location q , and a query distance r , find all the points within distance r from q .



Range Queries

- **Example** – find all the cafes within 1km distance of Wollongong central

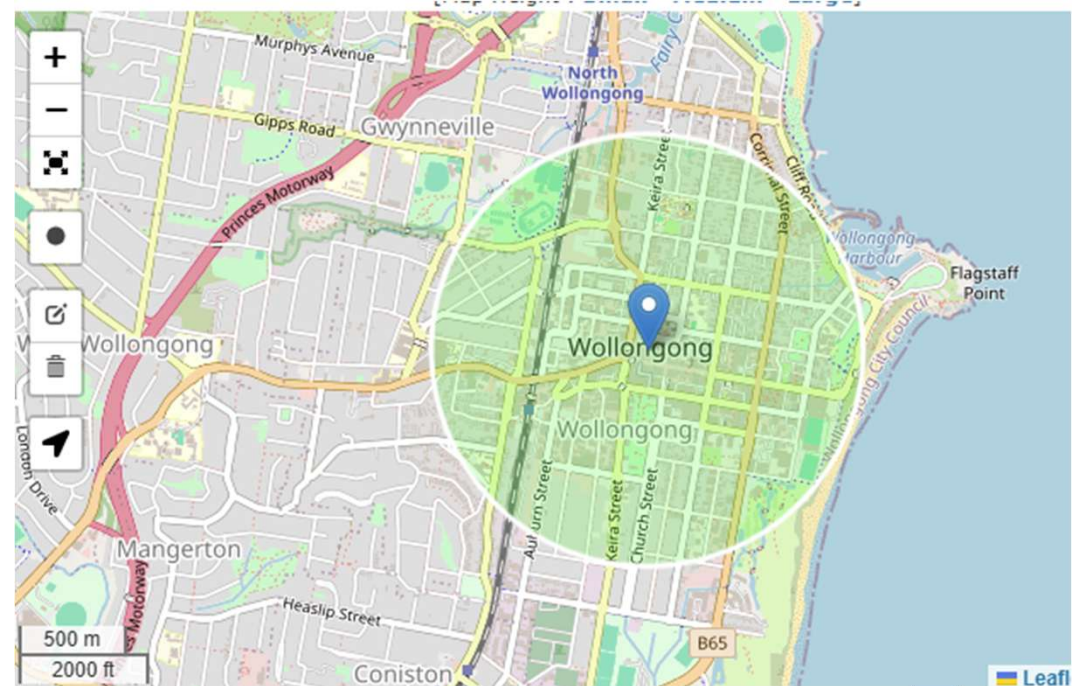
Add Radius

Radius Size km

Radius Center Point Choose one of:

1) Use the radius drawing tool on the map

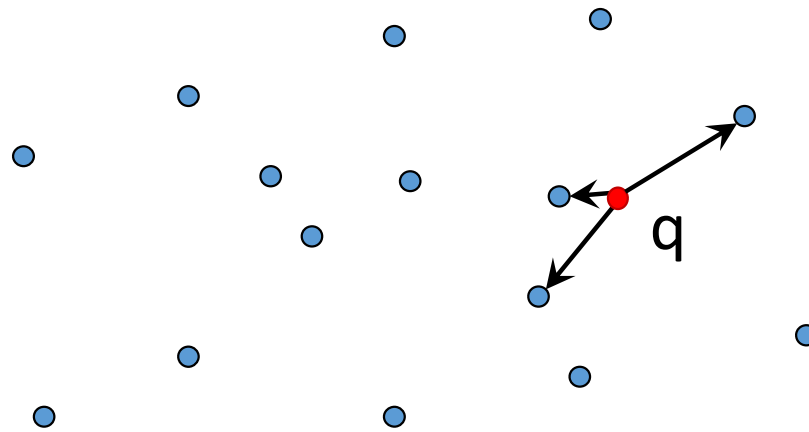
2) Place radius by location name :



*<https://www.freemaptools.com/>

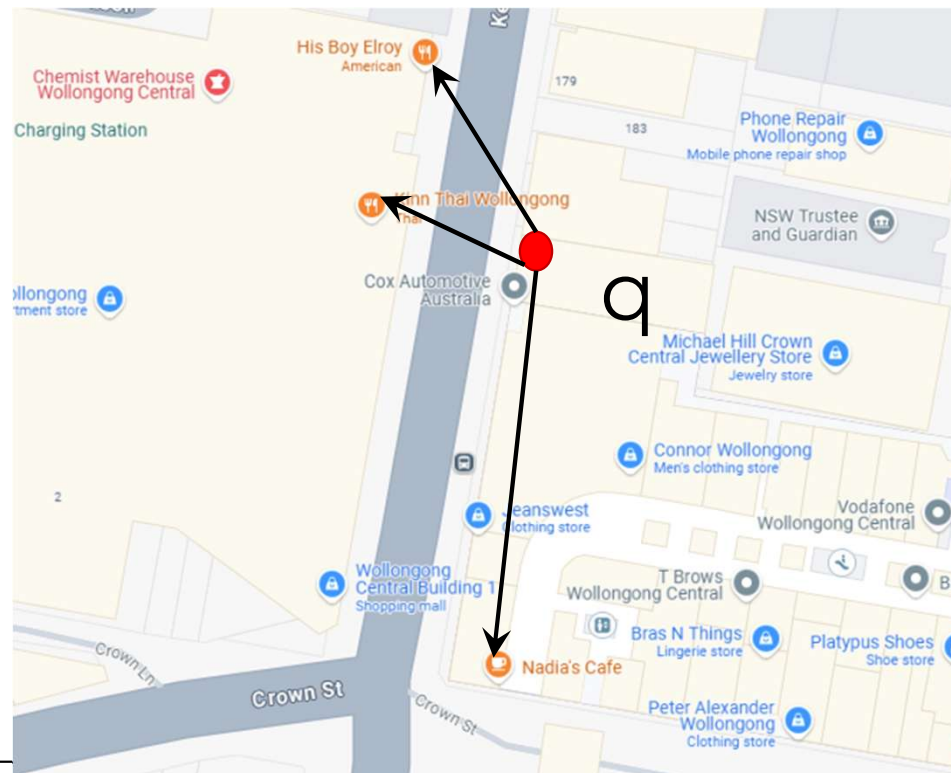
K nearest neighbor (kNN) queries

- Find k objects closest to the query location q .



K nearest neighbor (kNN) queries

- **Example** – find 3 restaurants nearest to my location

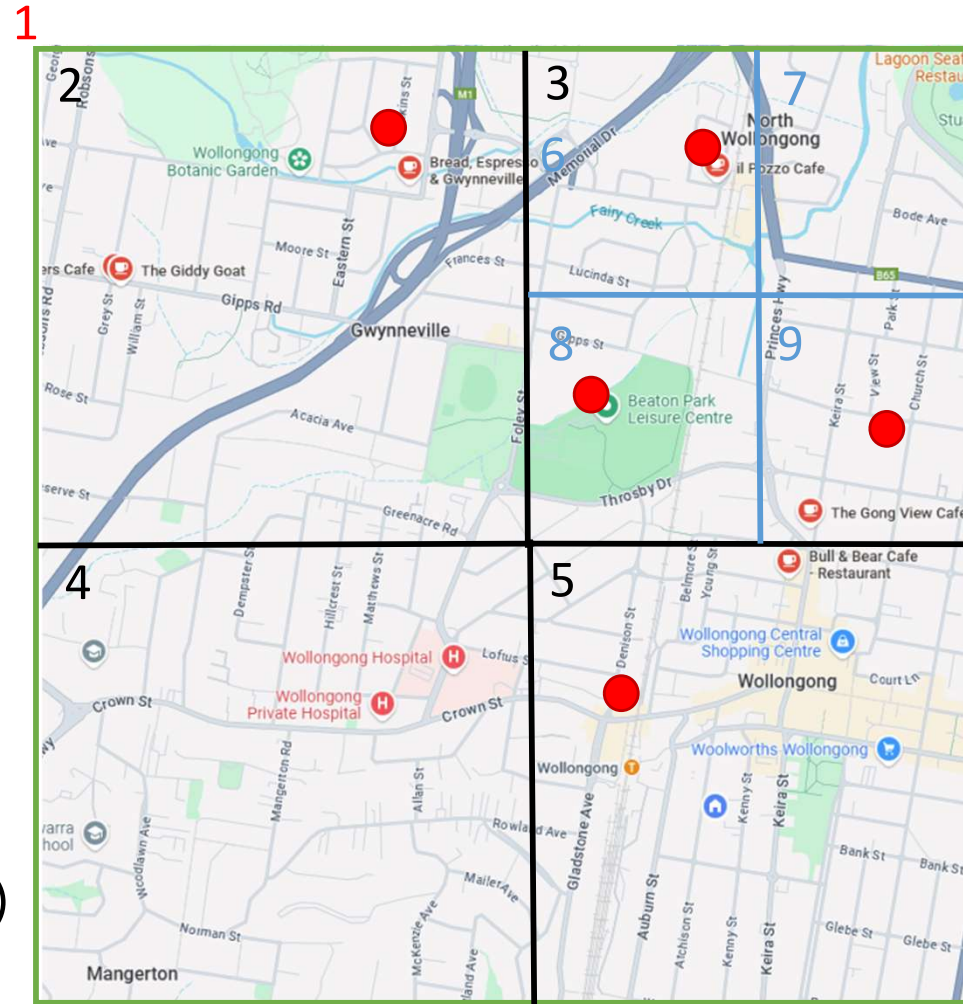
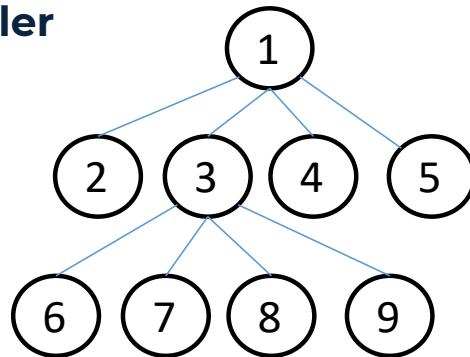
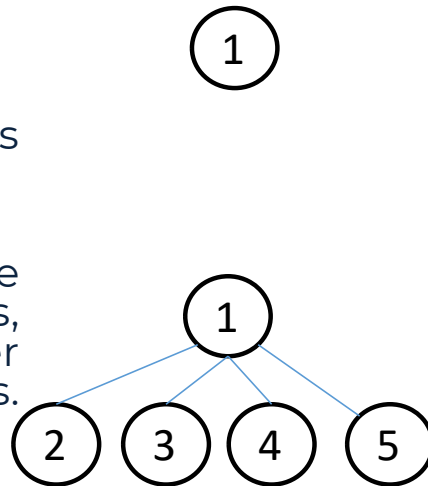


Spatial Index Structure

- **Naïve method**
 - scan every point and compute the distance, complexity: **$O(n)$**
- Organize the multidimensional data (point, line, region, etc.) to facilitate query processing
- **Hierarchical (tree-based) structures**
 - Quad-tree
 - R-tree

Quad-tree

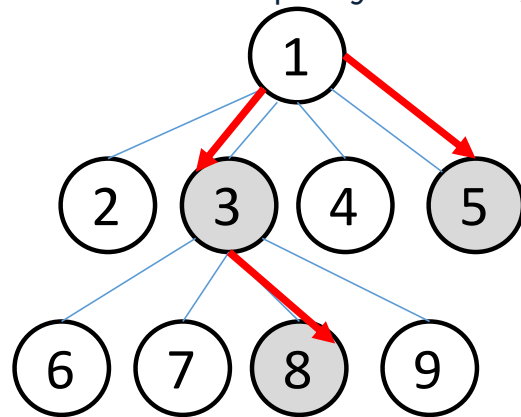
- Root node encompasses the entire space.
- If desired, subdivide along axis parallel lines, forming four smaller identical sized regions. (fanout=4)
- Some children may be subdivided further **until the capacity is smaller than a threshold**.



Quad-tree

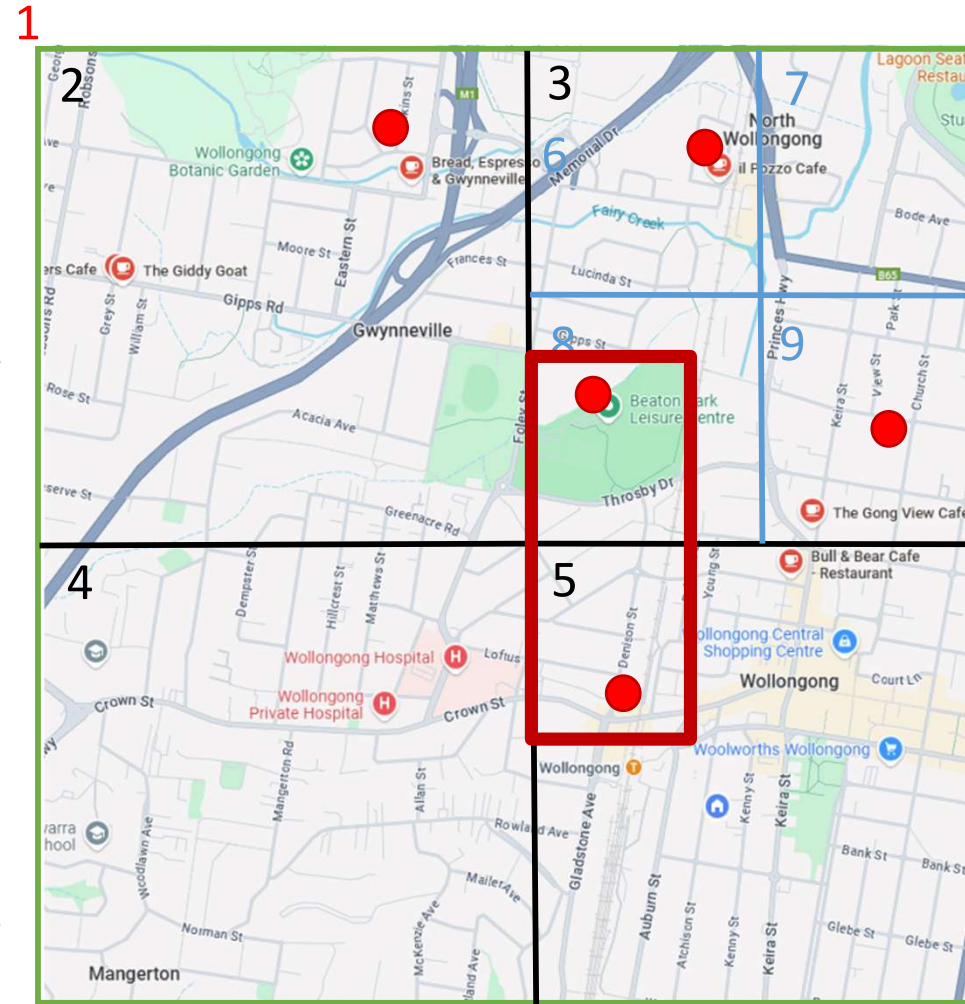
- **Window query processing**

- Filtering step: Find the branches that intersect with the query rectangle



- Refinement step: For each qualified leaf, check the original object against the query

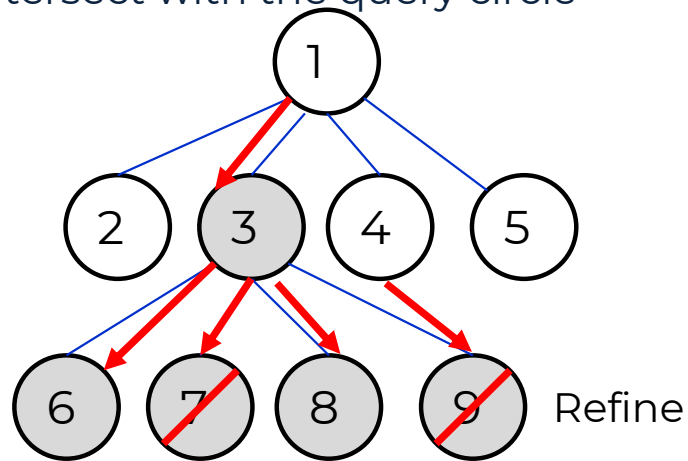
```
if (RectA.Left < RectB.Right && RectA.Right > RectB.Left &&  
RectA.Top > RectB.Bottom && RectA.Bottom < RectB.Top )
```



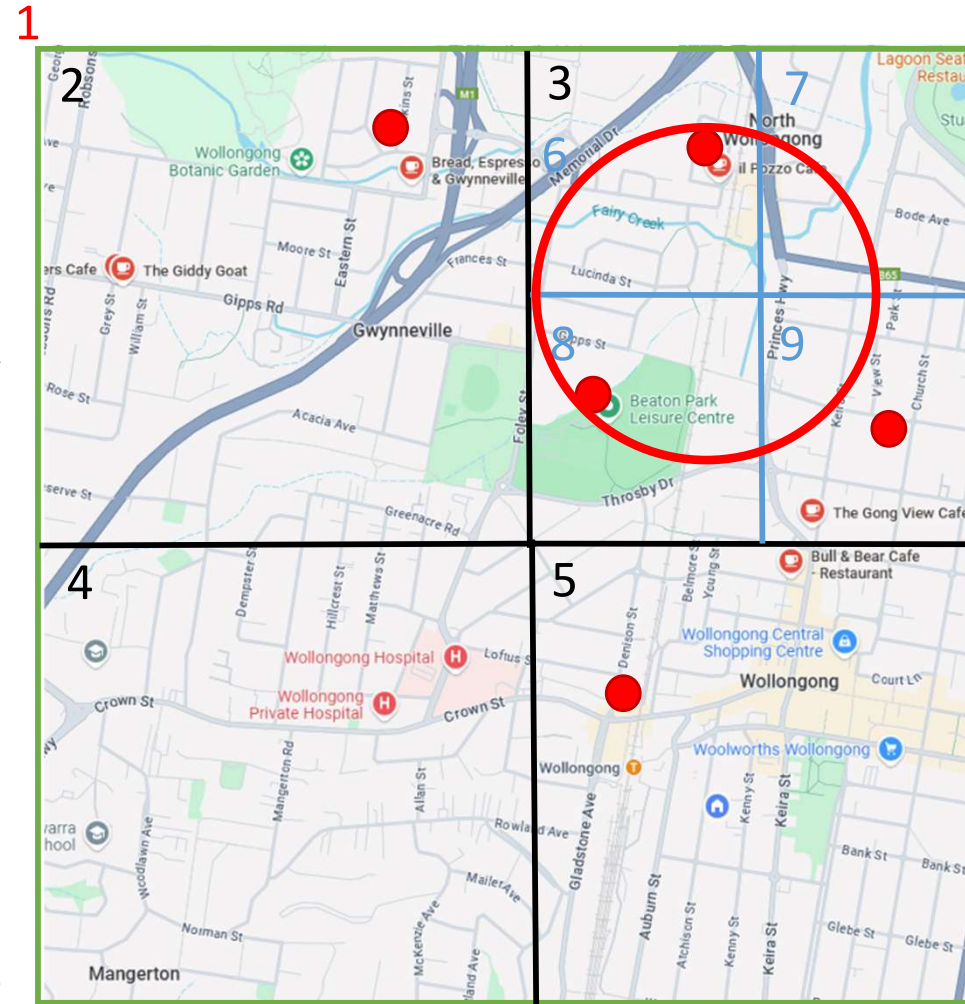
Quad-tree

- **Range query processing**

- Filtering step: Find the branches that intersect with the query circle



- Refinement step: For each qualified leaf, check the original object against the query

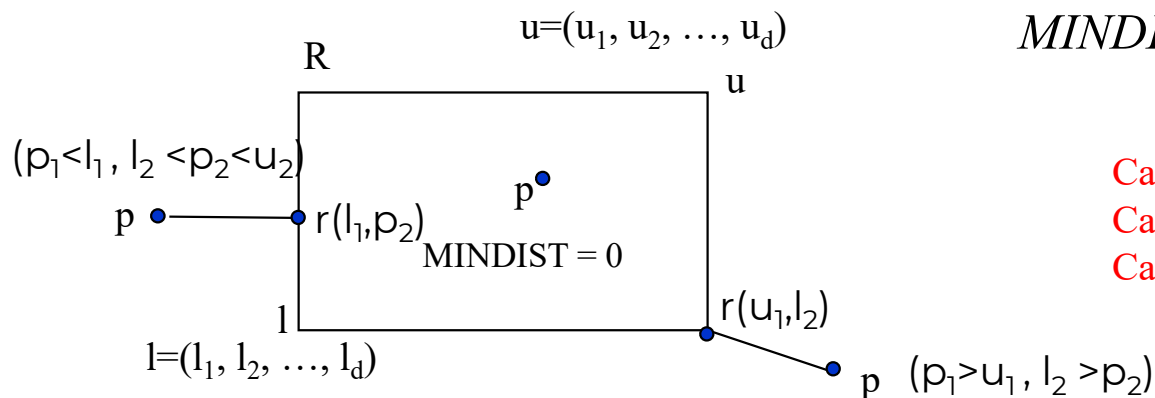


Background before kNN Query Processing

- Best first search using MINDIST
 - $\text{MINDIST}(q, R)$ is the minimum distance between a point q and a rectangle R
 - If the point is inside R , then $\text{MINDIST}=0$
 - If q is outside of R , MINDIST is the distance of q to the closest point of R (one point of the perimeter)

MINDIST computation

- MINDIST(p, R) is the minimum distance between p and R with corner points l and u
 - the closest point in R is at least this distance away



$$MINDIST(P, R) = \sqrt{\sum_{i=1}^d (p_i - r_i)^2}$$

Case 1: $r_i = l_i$ if $p_i < l_i$
 Case 2: $= u_i$ if $p_i > u_i$
 Case 3: $= p_i$ otherwise

Background before kNN Query Processing

- A priority queue is a data structure that stores elements along with a priority, and allows you to:
 - Insert elements with a given priority.
 - Remove (or access) the element with the highest priority (in our case, the smallest distance).
- In kNN search, we typically use a **min-priority queue**, where:
 - The item with the **lowest distance to the query** gets processed first.
 - This is why the search is called **best-first**—we always explore the most promising (closest) node next.

Background before kNN Query Processing

- **Enqueue (Insert into Priority Queue)**
 - **To enqueue** means to **insert an element** into the priority queue **with a priority**.
 - $\text{PriorityQueue} \leftarrow \text{Enqueue}(\text{Node}, \text{Priority})$
- **Dequeue (Remove from Priority Queue)**
 - **To dequeue** means to **remove and return the element** from the priority queue that has the **highest priority** (in our case, the *smallest* distance).
 - $\text{CurrentNode} \leftarrow \text{Dequeue}(\text{PriorityQueue})$

Best First Search for kNN Query

```
PriorityQueue  $\leftarrow$  Enqueue(root, MINDIST(root,q))
While(PriorityQueue not empty)
    CurrentNode  $\leftarrow$  Dequeue(PriorityQueue);
    if(k==0 & (MINDIST(CurrentNode,q)) > the lowest score in kNN)
        Break the while loop;
    else if (CurrentNode is Leaf )
        For each object o in CurrentNode
            if ( k>0) {add o into kNN and k=k-1;}
            else if (actualDistance(o,q) is better (smaller) than the lowest score in kNN)
                update kNN by replacing the lowest score object with o;
    else
        For each child of CurrentNode
            PriorityQueue  $\leftarrow$  Enqueue(child, MINDIST(child,q))
```

Best First Search for kNN Query

- Step 1: Initialize Priority Queue
 - Create **a priority queue (min-heap)** that will store nodes or entries ordered by their **minimum distance (MINDIST)** to the query point q .
 - Enqueue the root node of the index structure with its MINDIST to q .
- Step 2: Initialize kNN List
 - Create a list (or max-heap) to store the **k nearest neighbors** found so far.
 - Initially, this list is empty.

Best First Search for kNN Query

- Step 3: Explore Nodes in Order of Priority
 - While the priority queue is **not empty**:
 - Dequeue the node with the smallest MINDIST to q . This ensures we always explore the most promising node first (i.e., best-first strategy).
- Step 4: Prune Based on Current kNN
 - If we already found k neighbors, and the current node's MINDIST is **greater than the farthest distance** in our current kNN list:
 - **Terminate the search early** (prune the rest), as no better (closer) candidates can be found.

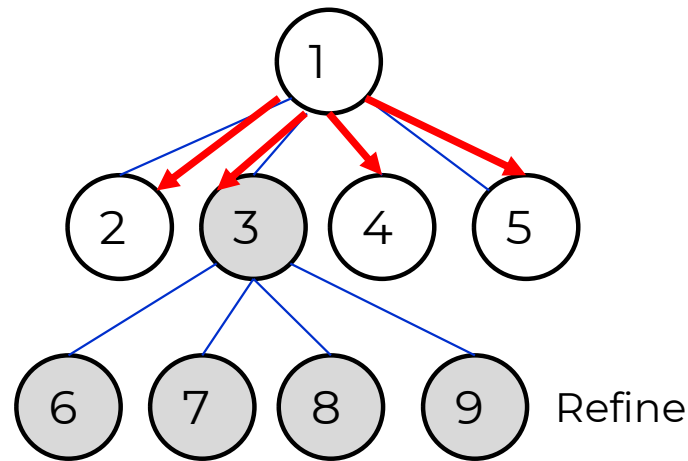
Best First Search for kNN Query

- Step 5: Process Leaf or Internal Node
 - If the current node is a **leaf node**:
 - For each object o in the leaf:
 - If fewer than k objects are in kNN, add o .
 - Else, compute $\text{actualDistance}(o, q)$ and if it's **smaller than the worst (farthest) in kNN**, replace the worst with o .
 - If the current node is an **internal (non-leaf) node**:
 - For each child node:
 - Compute $\text{MINDIST}(\text{child}, q)$, and **enqueue the child node** with this value.
- Step 6: Return Result
 - Once the queue is empty or pruning has occurred, the **kNN list contains the k nearest neighbors** to the query point q .

Quad-tree

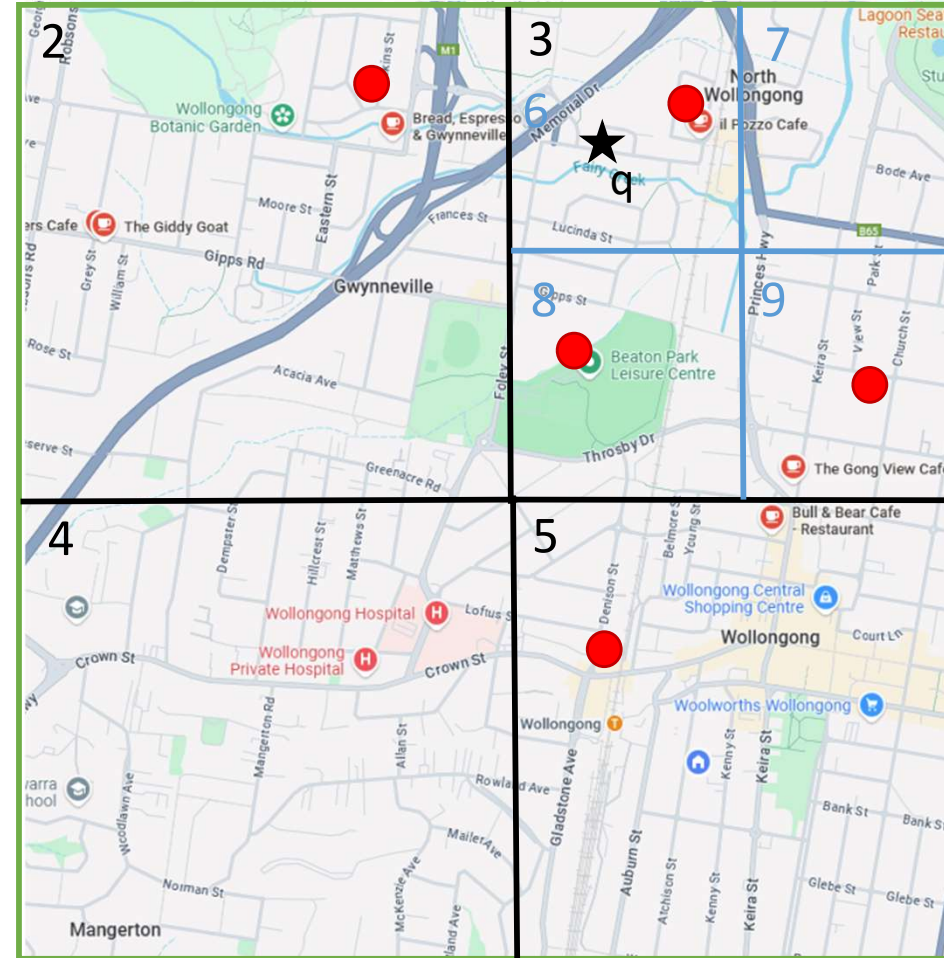
- kNN query processing**

- Filtering step: access branches based on MINDIST from q



- Step 1: dequeue: $1 \rightarrow 3, 2, 5, 4$
- Refinement step: For each qualified leaf, check the original object against the query ($\text{actualDistance}(q,o)$ for each o in currentnode)

1

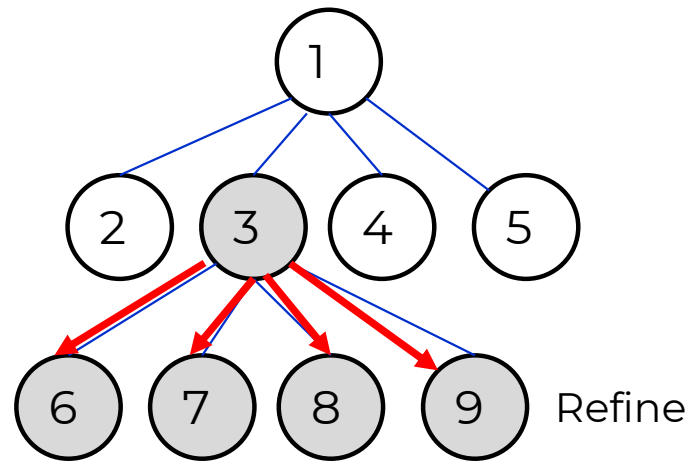


UNIVERSITY
OF WOLLONGONG
AUSTRALIA

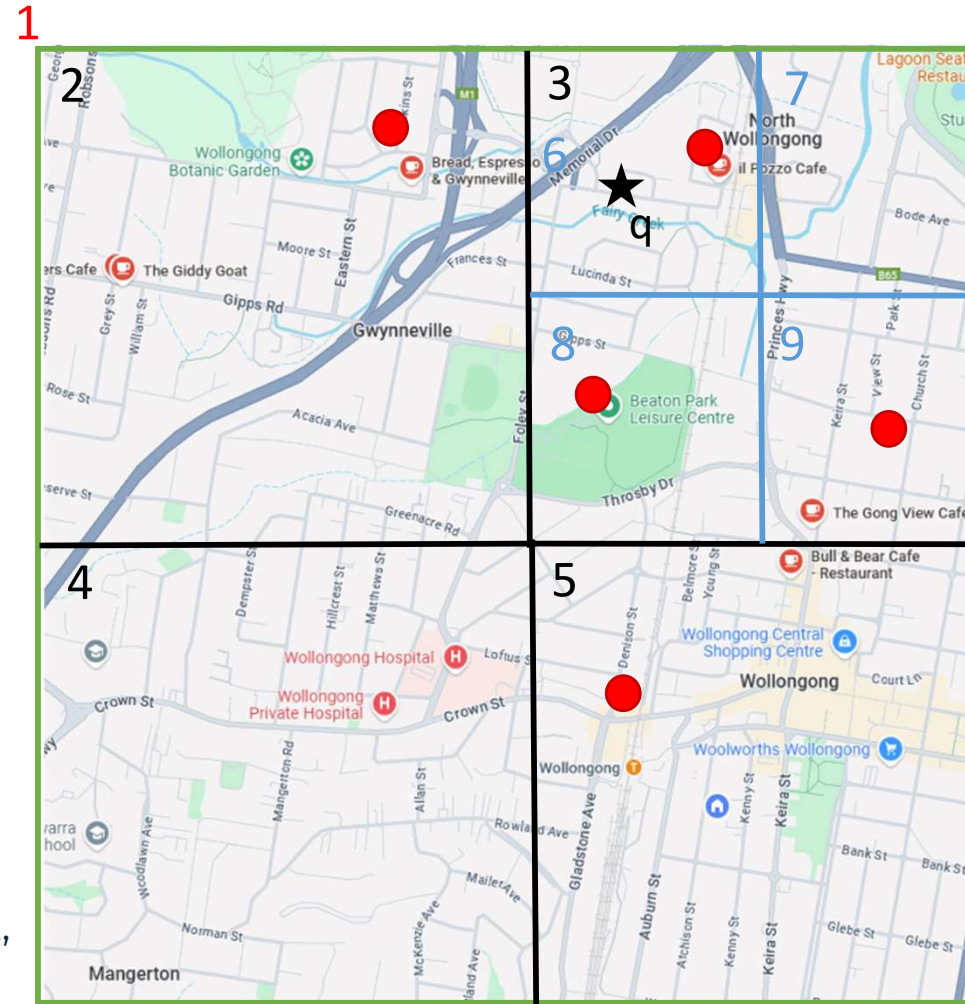
Quad-tree

- kNN query processing**

- Filtering step: access branches based on MINDIST from q



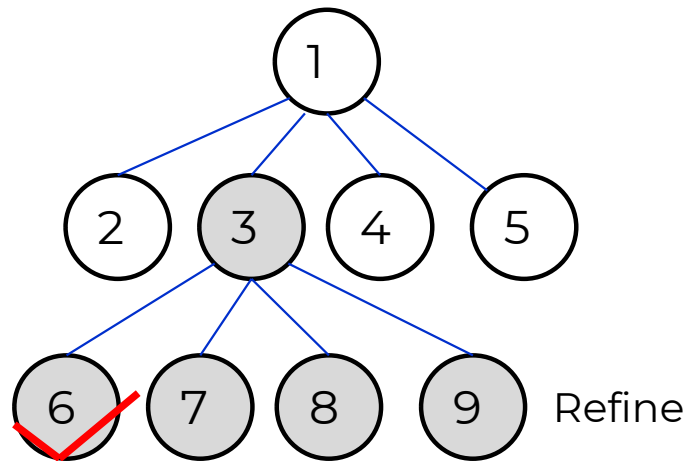
- Step 2: dequeue: 1 -> 3, 2, 5, 4 -> 6, 8, 7, 9, 2, 5, 4
- Refinement step: For each qualified leaf, check the original object against the query



Quad-tree

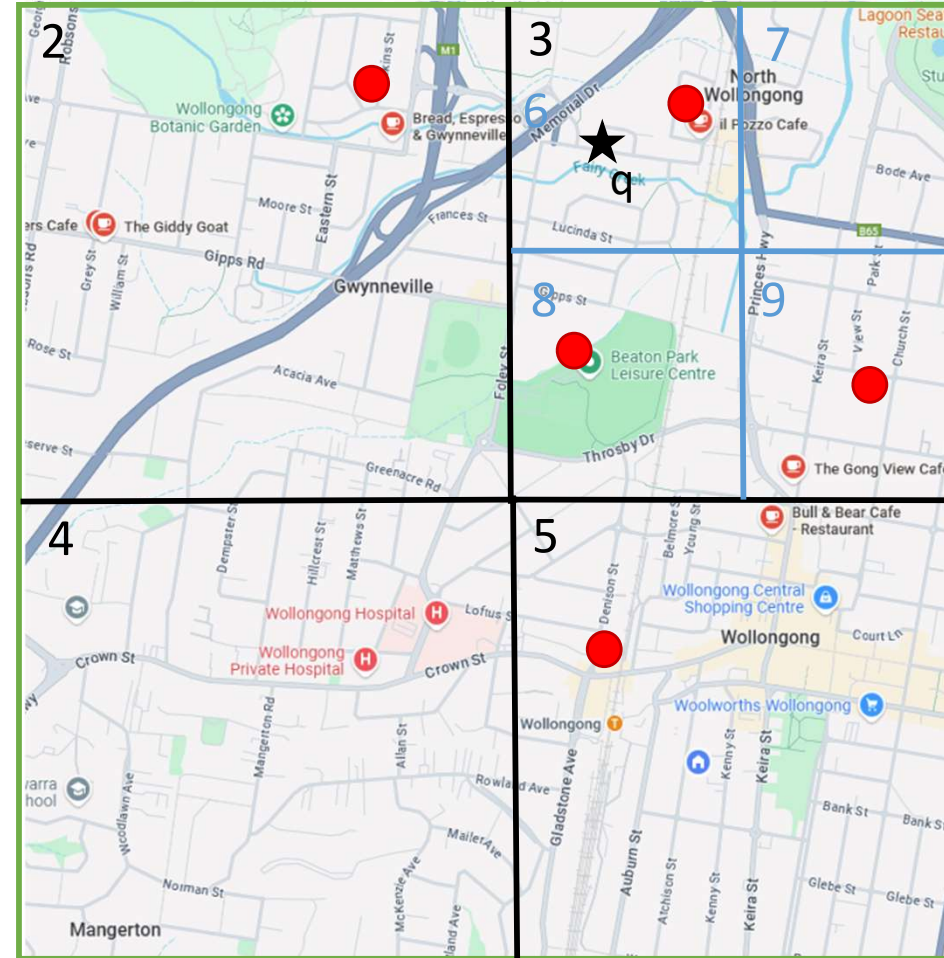
- kNN query processing**

- Filtering step: access branches based on MINDIST from q



- Step 3: process each leaf node:
- ~~6~~ 8, 7, 9, 2, 5, 4
- Refinement step: For each qualified leaf, check the original object against the query

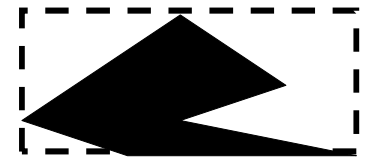
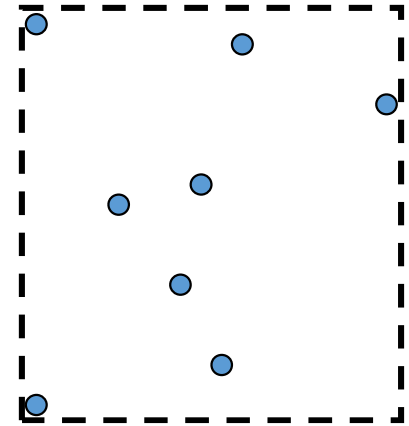
1



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

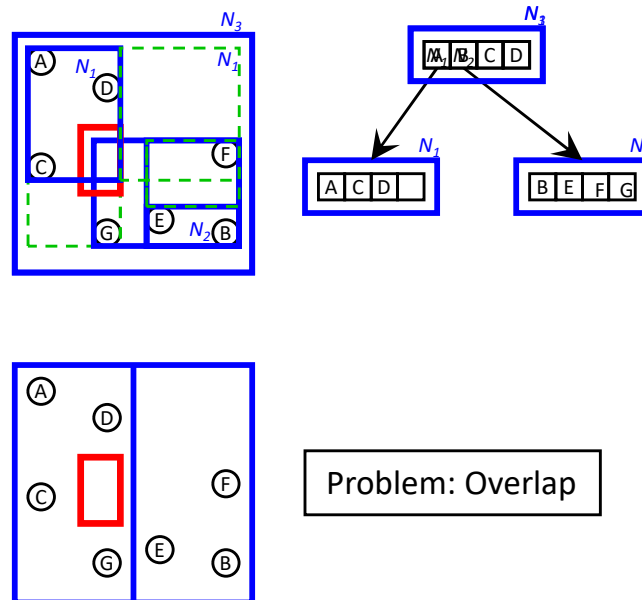
R-tree

- Idea: group nearby objects and organize them with minimum bounding rectangles (MBR)
- An MBR is a bounding box of the objects with the minimum size
- R-tree is a **balanced** tree
 - Uniform dataset (Quad-tree is balanced)
 - **Skewed dataset**
- Can index any shape of data, by representing as MBR
- An R-tree node has a max capacity f , and a min occupancy (usually $f/2$)



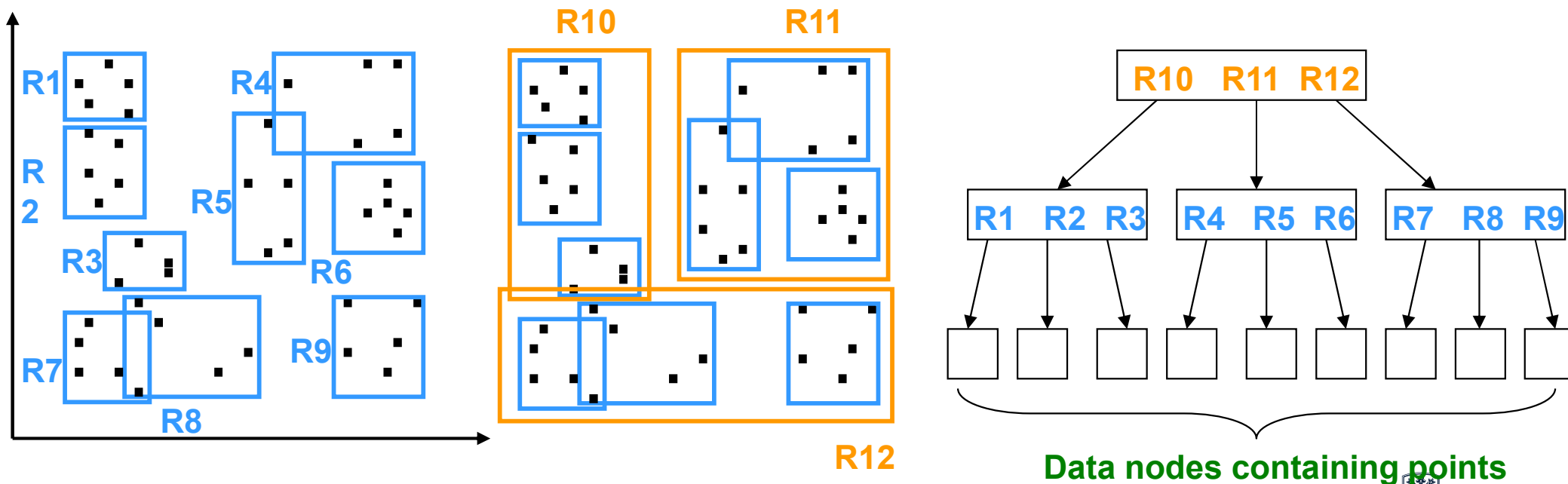
R-tree Construction: Data Insertion

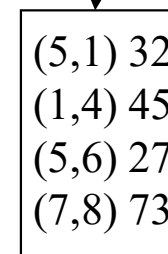
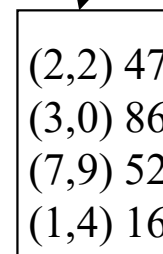
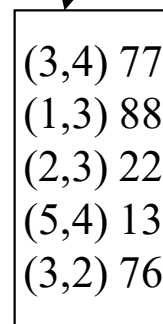
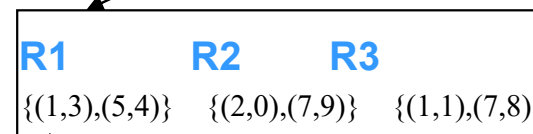
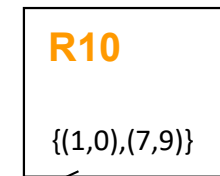
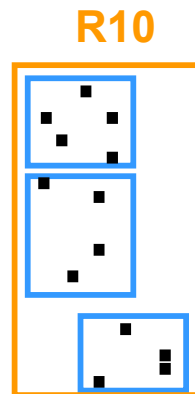
- Split a node if capacity exceeds ($f = 4$ in the example)
- Where to insert the new object $\rightarrow$ Find the entry that needs the least enlargement to include the new object



R-tree Construction

- Construct the R-tree in a bottom up fashion (fanout $f=6$ in the example)





Data nodes

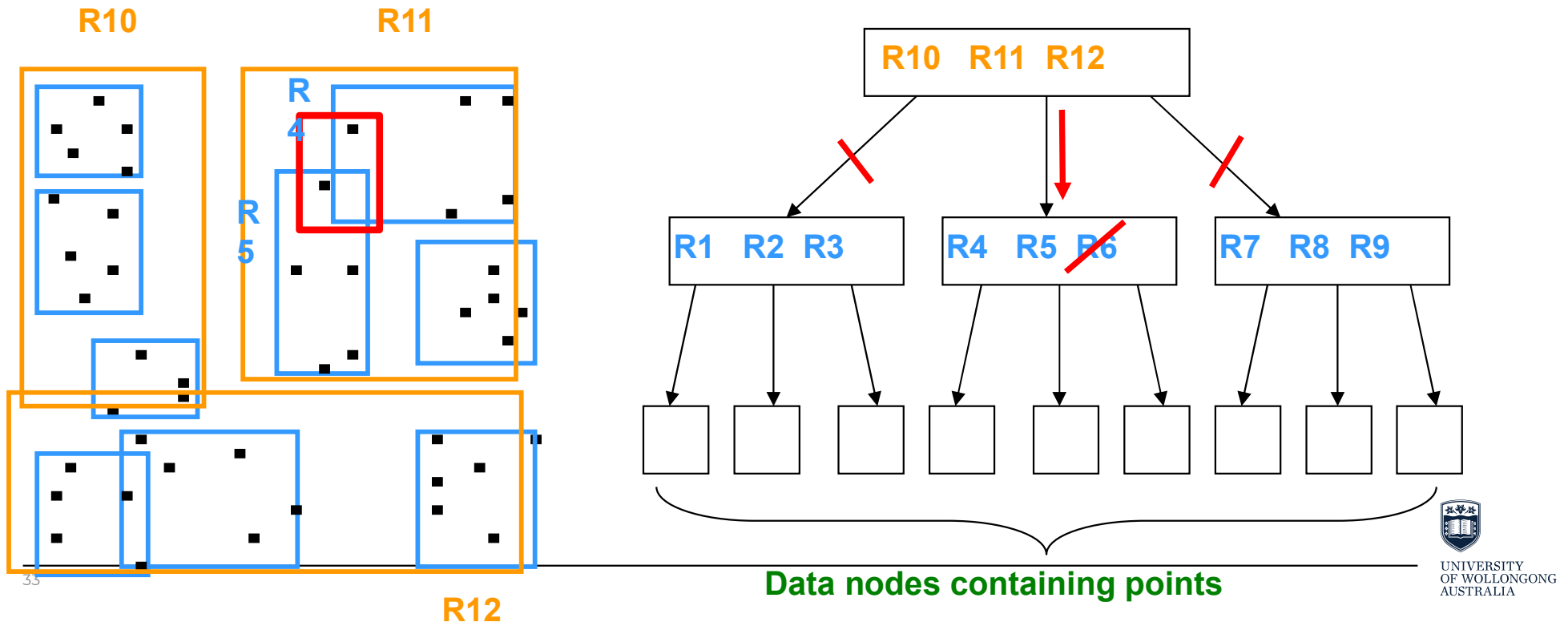
At the leaf nodes we have the location, and a pointer to the record in question.

Data record: (latitude,longitude)
object_id

At the internal nodes, we just have MBR information.

R-tree - Window query processing

- Filtering step: Find the branches that intersect with the query rectangle; Refinement step



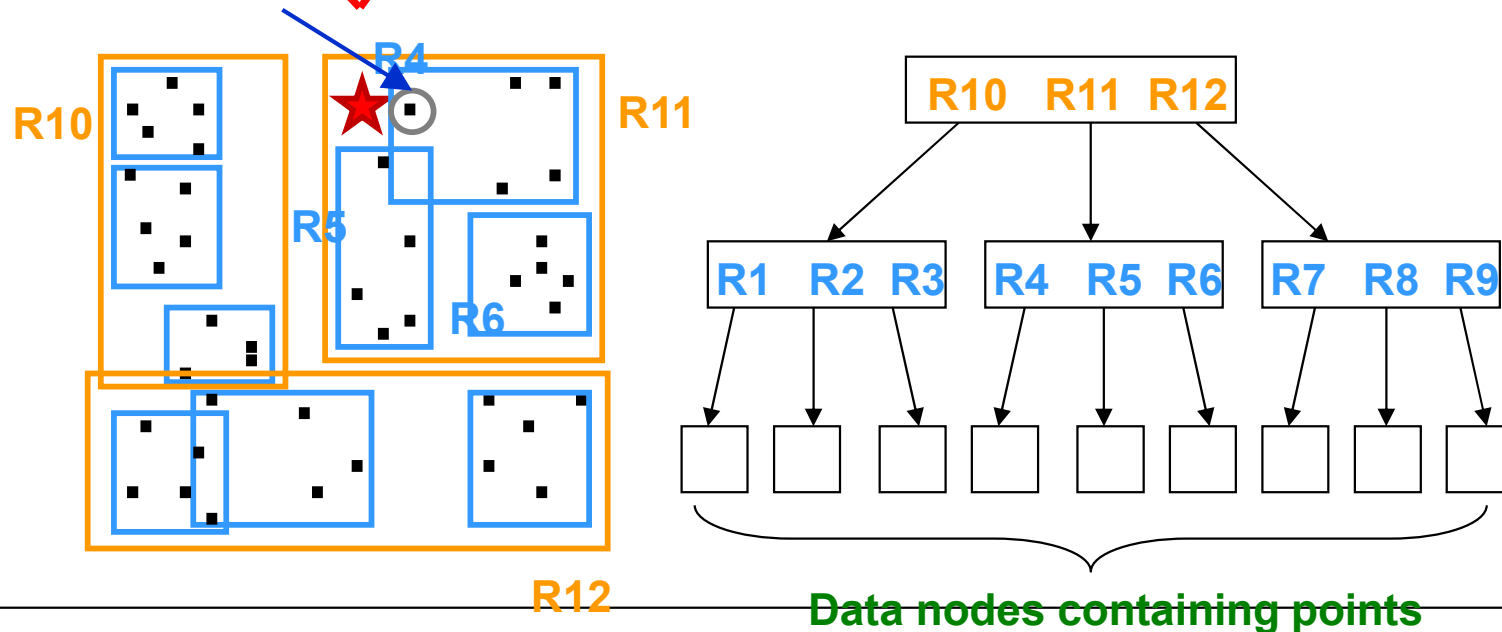
```

public void windowQuery(Region q, RTree tree)
{
    PriorityQueue queue = new PriorityQueue(100, new NNEntryComparator());
    //start from root node
    RtreeEntry e = new RtreeEntry(tree.m_rootID, false);
    queue.add(new NNEntry(e, 0.0));
    int count = 0;
    while (queue.size() != 0){
        NNEntry first = (NNEntry) queue.poll();
        e = (RtreeEntry)first.m_pEntry;
        //if the node is a data object, check if it's inside the query region, if yes --> result
        if(e.isLeafEntry){
            Region obj = (Region)e.getShape();
            if(q.contains(obj))
            {
                System.out.println(e.getIdentifier() + " " + obj.m_pLow[0]+ " " +obj.m_pLow[1]);
                count++;
            }
        }
        else{
            Node n = tree.readNode(e.getIdentifier());
            for (int cChild = 0; cChild < n.m_children; cChild++)
            {
                //check for each children if that intersects/contains in the query region. if yes --> add to the queue.
                if(q.contains(n.m_pMBR[cChild]) || q.intersects(n.m_pMBR[cChild]))
                {
                    if (n.m_level == 0)
                        e = new RtreeEntry(n.m_pIdentifier[cChild],n.m_pMBR[cChild], true);
                    else
                        e = new RtreeEntry(n.m_pIdentifier[cChild],n.m_pMBR[cChild],false);
                    queue.add(new NNEntry(e, 0));
                }
            }
        }
    }
}

```

R-tree - kNN query processing

- Best first search: start with the root node
- Maintain a priority queue of the nodes based on minDist from q
- ~~R11~~, R10, R12 → ~~R4~~, R5, R6, R10, R12



Complexity comparison

	Query
Naïve scanning	$O(n)$
Quad-tree	$O(\log_4 n)$
R-tree	$O(\log_f n)$

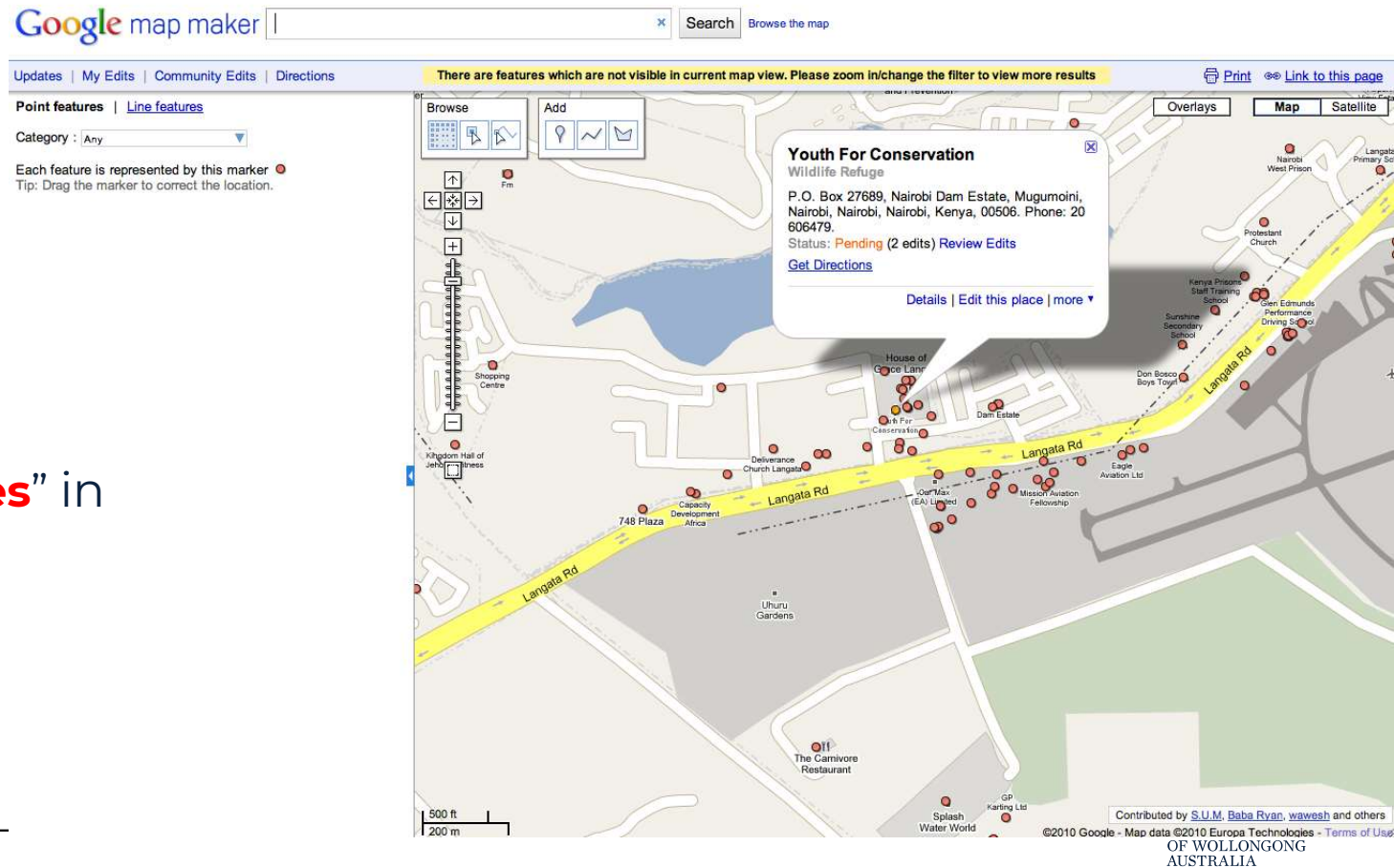
Kothuri Venkata Ravi Kanth, Siva Ravada, Daniel Abugov: Quadtree and R-tree indexes in oracle spatial: a comparison using GIS data. SIGMOD Conference 2002: 546-557

You Jung Kim, Jignesh M. Patel: Performance Comparison of the R*-Tree and the Quadtree for kNN and Distance Join Queries. IEEE Trans. Knowl. Data Eng. 22(7): 1014-1027 (2010)

<http://informix-spatial-technology.blogspot.com.au/2012/01/comparison-on-b-tree-r-tree-quad-tree.html>

Spatial-textual Data

- Data with
 - Location
 - Text
- Query example
 - find all the “cafes” in **Wollongong**



Spatial-Textual Index

- Spatial index + Textual index
- Example: The Inverted R-tree (IR-tree) → R-tree + Inverted file

Inverted file

- Construct a vocabulary list (remove stop words, stemming terms)
- For each term T , store a list of all document IDs that contain T .

Doc 1

The house
has a
garden.

Doc 2

There are many beautiful
flowers in the garden of the
house.

Doc 3

The garden has many
beautiful flowers.

Vocabulary

garden
beautiful
flowers
house

Occurrences

3
2
2
2

garden → 1, 2, 3

beautiful → 2, 3

flowers → 2, 3

house → 1, 2

Inverted list of a term

Spatial-Textual Queries

- Window/range + Boolean queries
 - Find all the objects in Wollongong CBD containing keywords “coffee” and “sandwich”
 - Find all the objects in Wollongong CBD containing keywords “coffee” or “juice”
- Spatial-textual kNN queries: Find k objects with the **highest score** based on both location and keywords “coffee”, “sandwich”

Spatial-Textual Score

- Spatial textual score of an object from a query

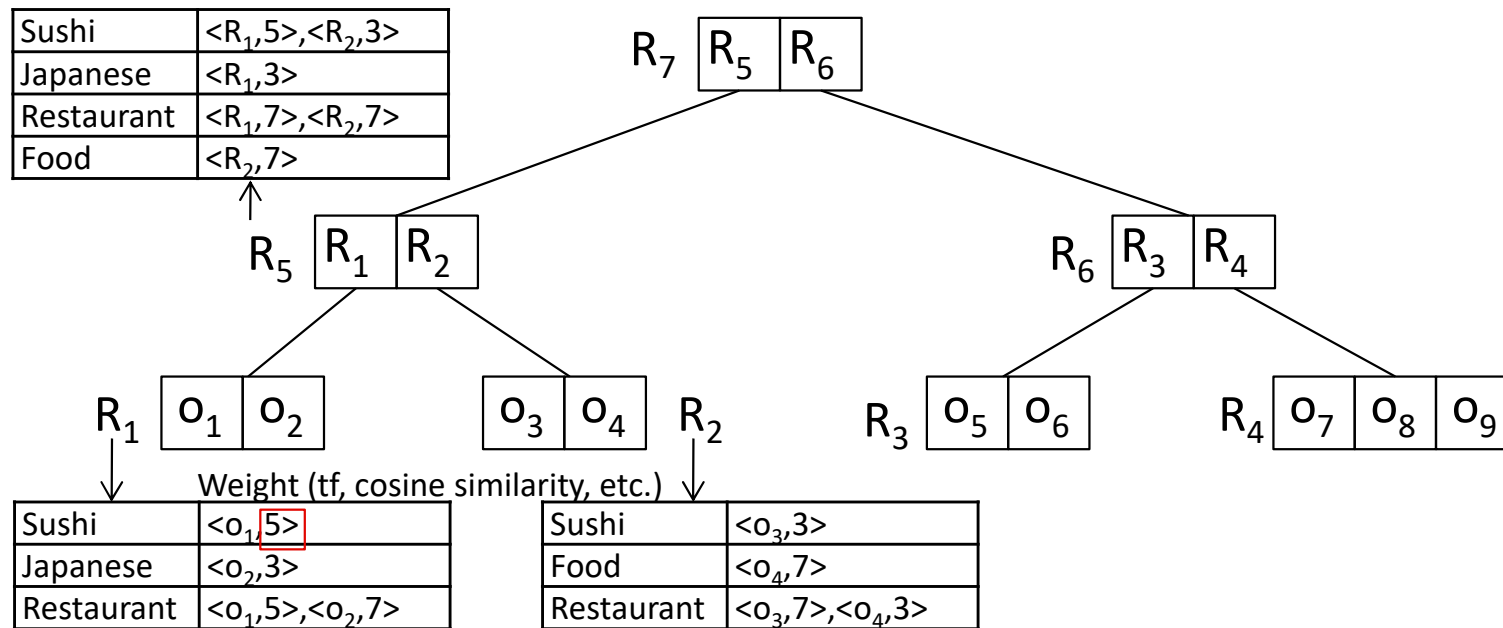
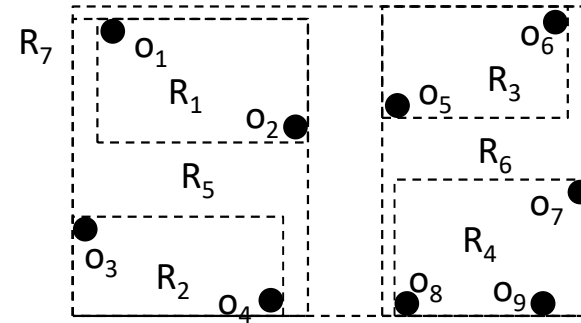
$$\alpha \times S(o, q) + (1 - \alpha) \times T(o, q)$$

Diagram illustrating the components of the Spatial-Textual Score formula:

- α : Preference weight [0,1]
- $S(o, q)$: Spatial proximity
- $T(o, q)$: Textual score

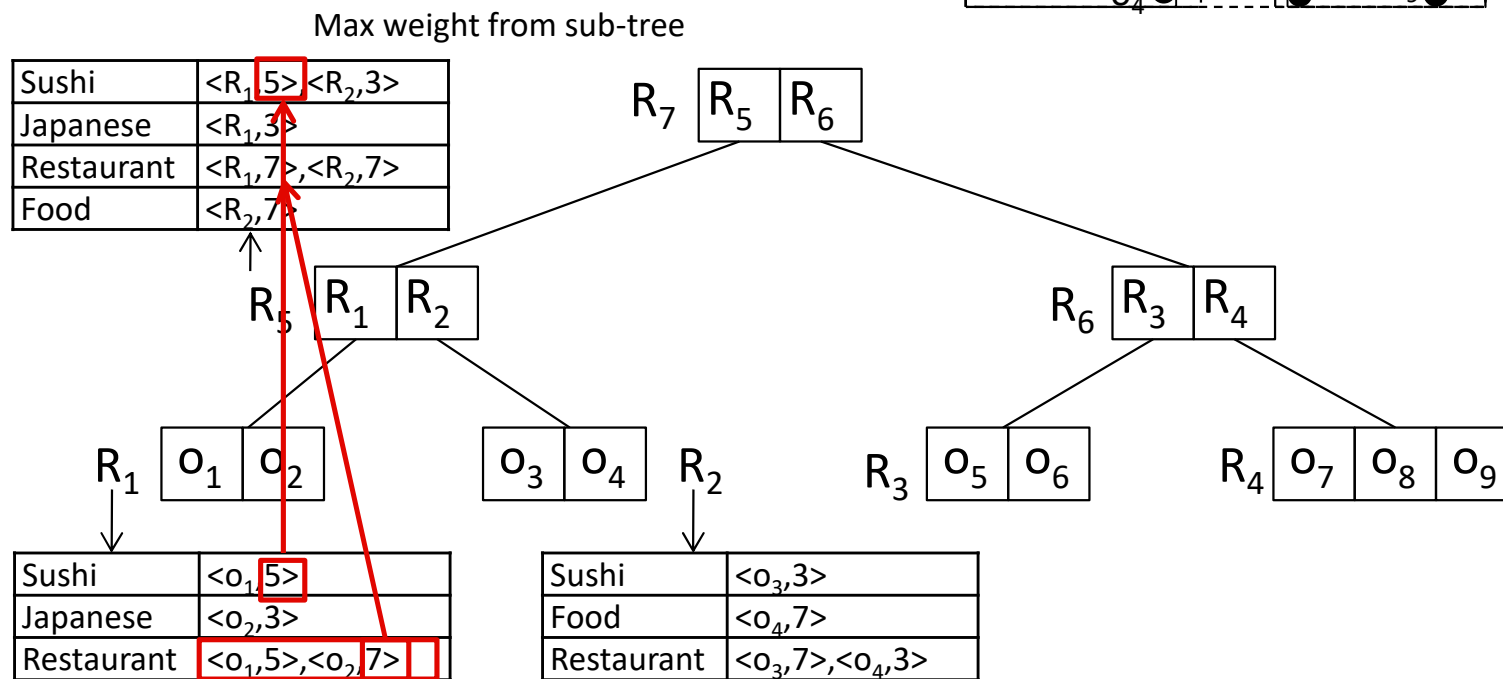
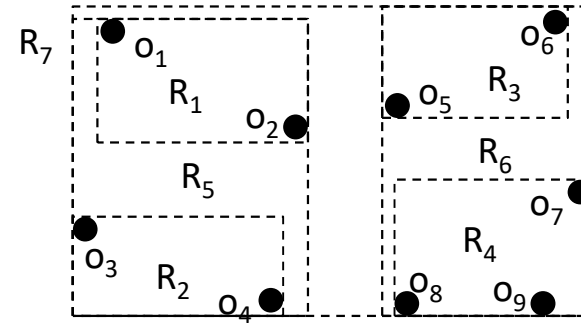
IR-tree

A combination of R-trees and
Inverted files



IR-tree

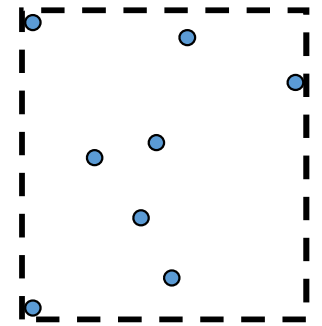
The IR-tree augments each node of the R-tree with a summary of the text content of the objects in the corresponding subtree.



KNN Search and Similarity Function

- Spatial textual score (minScore) of a node of the IR-tree from a query

$$\alpha \times \text{Spatial Score}(R, q) + (1 - \alpha) \times \text{Textual Score}(o, q)$$



- **Best first search:** start with the root node
 - Same as spatial-only knn search method, except that the minDist function (in spatial-only knn search) is **replaced by minScore function above**
- Maintain a priority queue of the nodes based on minScore from q

Textual Similarity Score

- **TF** stands for **Term Frequency**, and it measures **how often a word appears in a document**.

$$\text{TF}(w, d) = \frac{\text{Number of times word } w \text{ appears in document } d}{\text{Total number of words in document } d}$$

- Example:
 - Let's say we have a document: "**apple banana apple**"
 - $\text{TF}(\text{"apple"}) = 2 / 3 = 0.67$
 - $\text{TF}(\text{"banana"}) = 1 / 3 = 0.33$

Textual Similarity Score

- **Cosine similarity** measures how **similar two documents (or word vectors)** are by calculating the **cosine of the angle** between them.

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

- $A \cdot B$ is the **dot product**
 - $\|A\|$ and $\|B\|$ are the **magnitudes** (lengths) of the vectors
-
- Example: Two Documents
 - **Doc1**: "apple banana apple" → TF vector = [0.67, 0.33]
 - **Doc2**: "banana banana apple" → TF vector = [0.33, 0.67]
 - Vectors: $A = [0.67, 0.33]$ $B = [0.33, 0.67]$

Textual Similarity Score

- Vectors: $A = [0.67, 0.33]$ $B = [0.33, 0.67]$

Step-by-Step Cosine Calculation:

1. Dot Product:

$$A \cdot B = (0.67 \times 0.33) + (0.33 \times 0.67) = 0.2211 + 0.2211 = 0.4422$$

2. Magnitudes:

$$\|A\| = \sqrt{0.67^2 + 0.33^2} = \sqrt{0.4489 + 0.1089} = \sqrt{0.5578} \approx 0.75$$

$$\|B\| = \sqrt{0.33^2 + 0.67^2} = \sqrt{0.1089 + 0.4489} = \sqrt{0.5578} \approx 0.75$$

3. Cosine Similarity:

$$\text{Cosine}(A, B) = \frac{0.4422}{0.75 \times 0.75} = \frac{0.4422}{0.5625} \approx 0.786$$

Applications

- Real-time taxi calling service

Passenger is the task requester



The task is to pick up the passenger at location A and send her/him to location B



Applications

- Real-time taxi calling service

A typical dynamic matching problem

dynamically appear on the platform



Uber is the platform, and its core concern is how to assign taxis to pick up different passengers efficiently

Applications

- Real-time taxi calling service

**Task
Assignment**



**Incentive
Mechanism**

**Quality
Control**

**Privacy
Protection**

Applications

- Food delivery service

User is the task requester



The task is to pick up the food and send the food to the user



Applications

- Food delivery service

A typical dynamic planning problem

Orders and available deliverers dynamically appear on the platform



Platform



Grubhub is the platform, and its core concern is how to design effective plans for the deliverers

Applications

- Food delivery service

**Task
Assignment**

**Orders and available deliverers
dynamically appear on the platform**



Platform



**Incentive
Mechanism**

**Quality
Control**

**Privacy
Protection**

Applications

- Ridesharing Service



Graph Data and Computing



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Logistics

- (1) Advanced Database Overview
- (2) Spatial Data and Computing
- (3) Graph Data and Computing
- (4) Text Data and Processing

Multi-Modal Database



- (1) AI4SE: artificial intelligence for software engineering
- (2) Software Engineering in Cloud Computing I
- (3) Software Engineering in Cloud Computing II

Software



- (1) IoT and Edge Computing
- (2) Digital Twin Technology
- (3) Blockchains and Smart Contracts
- (4) Emerging Cyber-Attacks

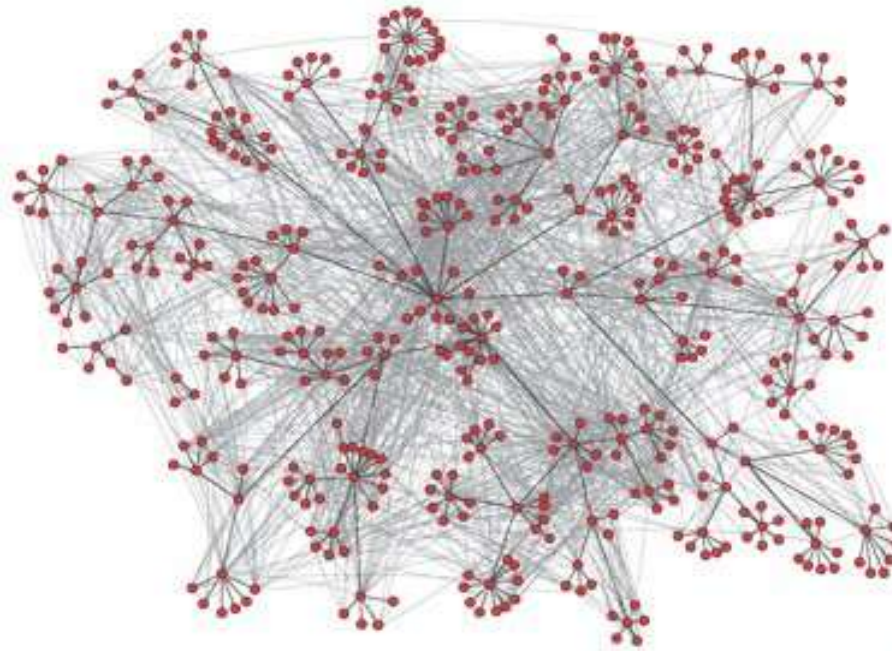
Networking



Outline

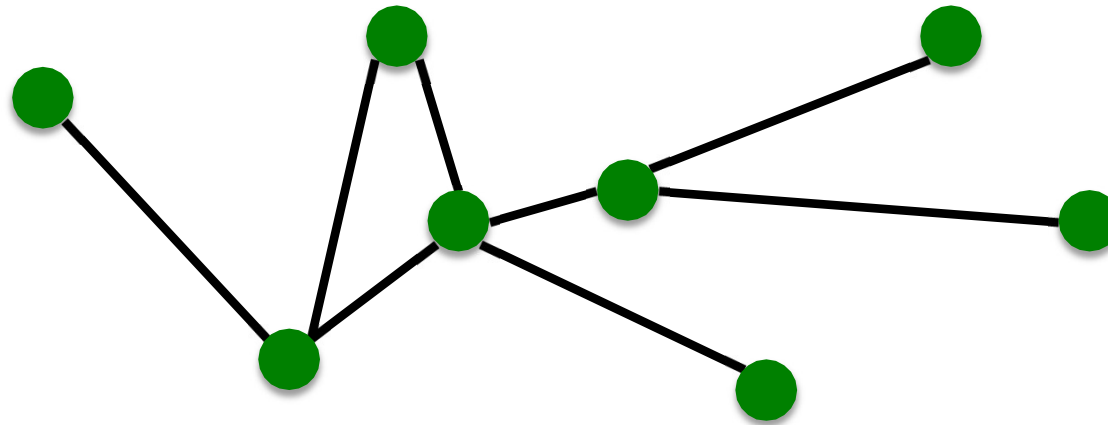
- Graph Structure
- Graph Representation
- Network Properties
- Application: Influence Maximization in Networks

Structure of Networks



A network is a collection of objects where some pairs of objects are connected by links **What is the structure of the network?**

Components of a Network



- **Objects:** nodes, vertices
- **Interactions:** links, edges
- **System:** network, graph

N

E

$G(N,E)$

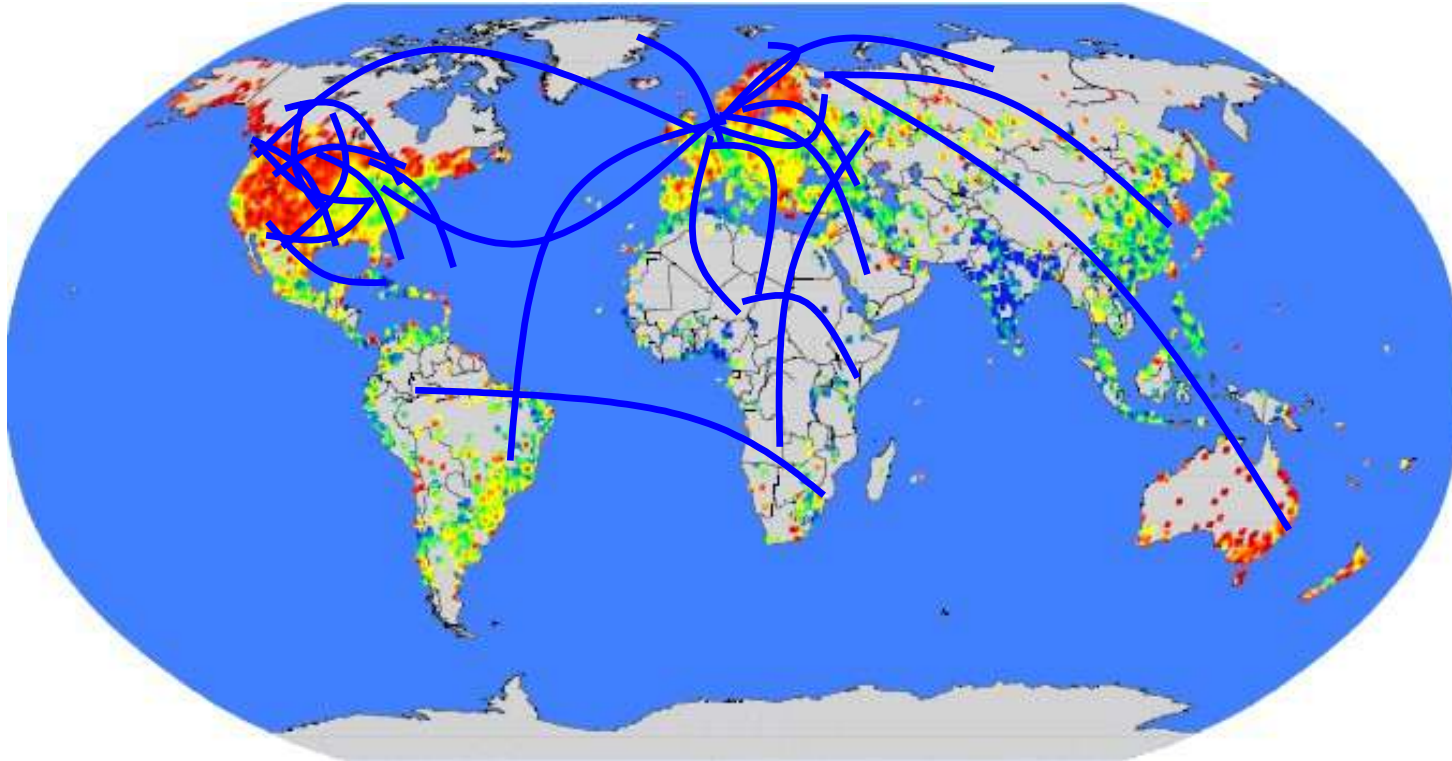


Example – MSN Messenger

- 1 month activity
 - 245 million users logged in
 - 180 million users engaged in conversations
 - More than 30 billion conversations
 - More than 255 billion exchanged messages

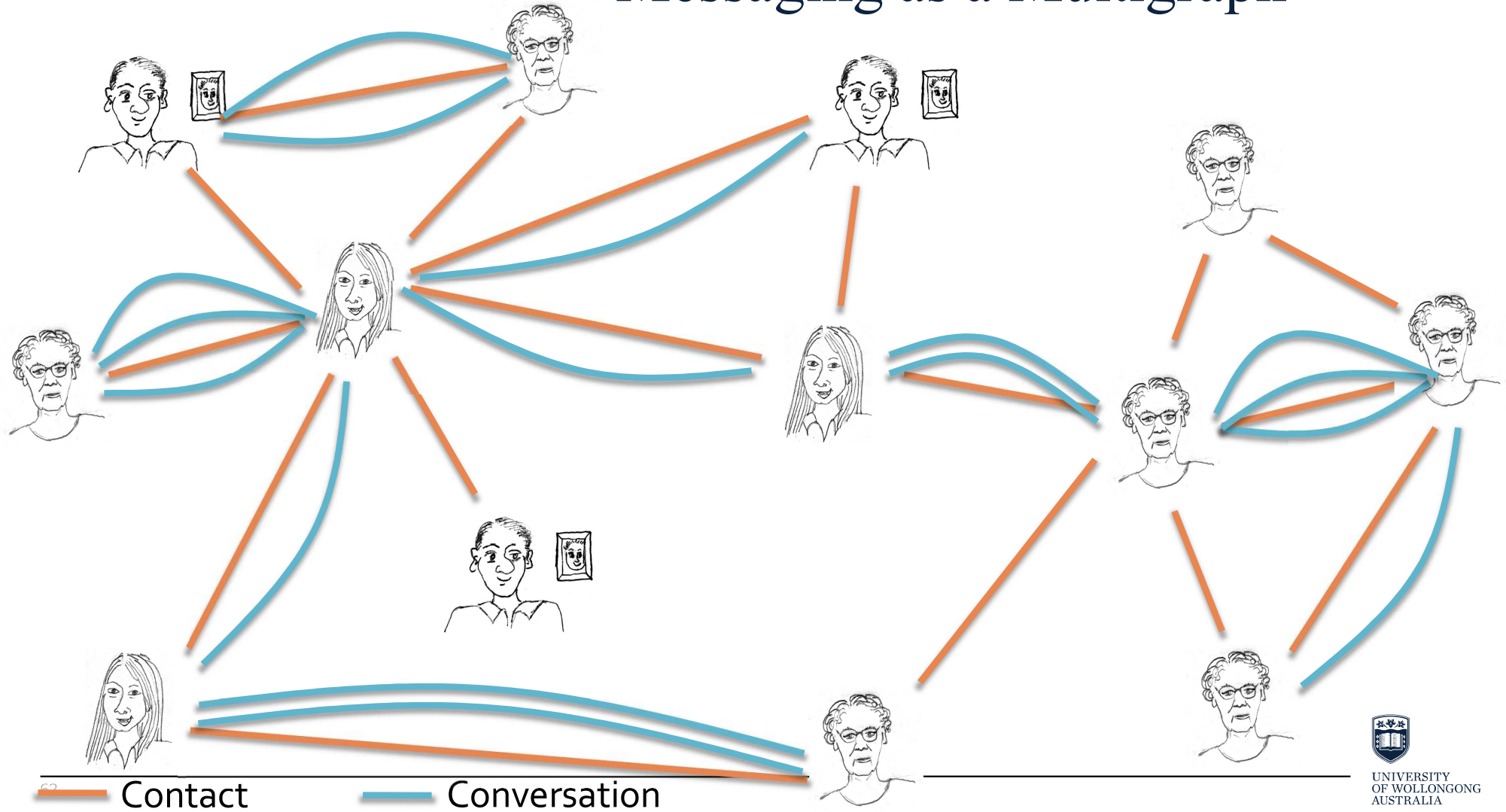


MSN Communication Network



Network: 180M people, 1.3B edges

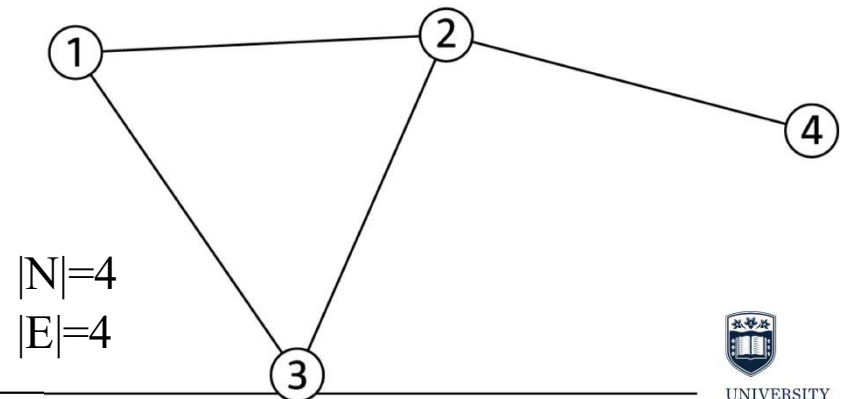
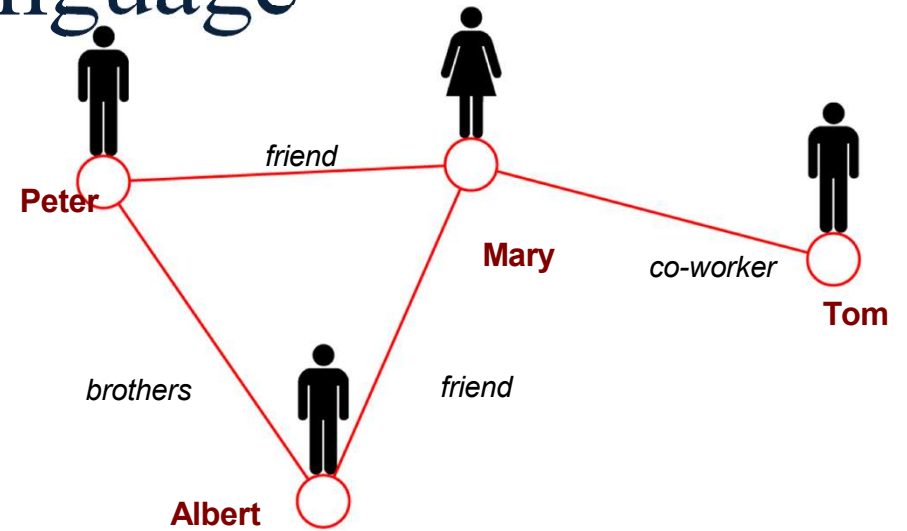
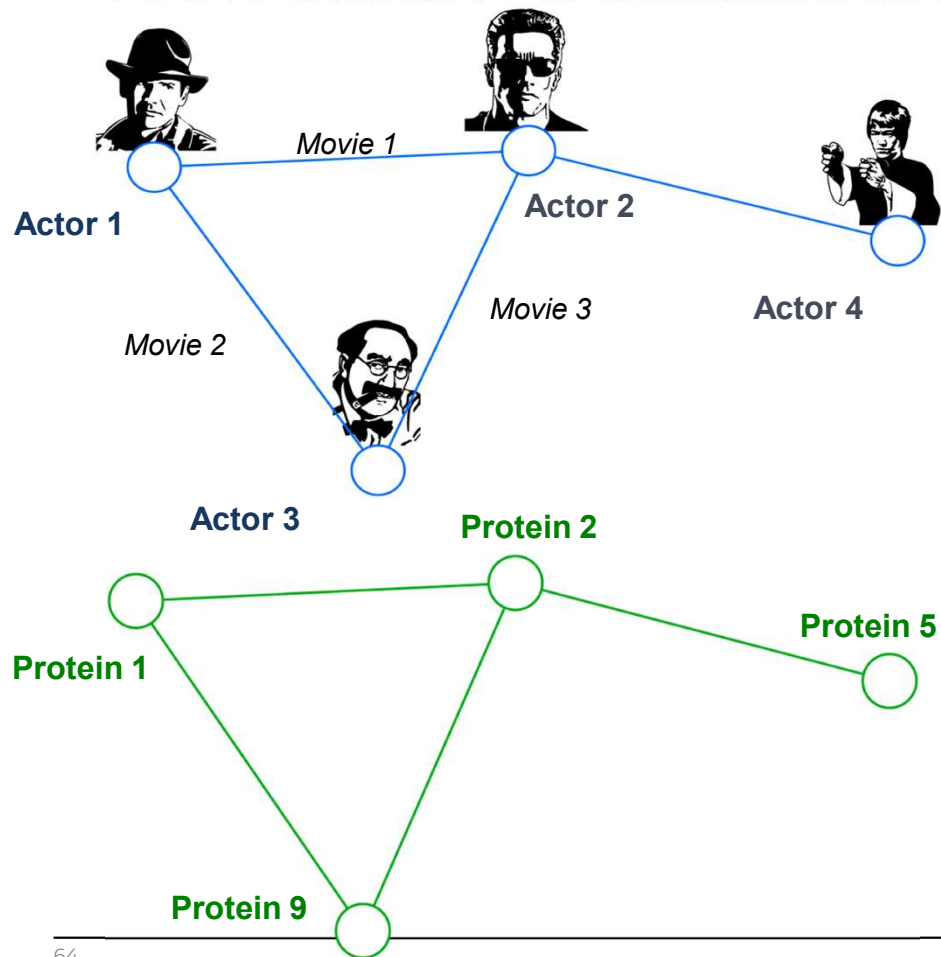
Messaging as a Multigraph



Networks or Graphs

- **Network** often refers to real systems
 - Web, Social network, Metabolic network
- Language: Network, node, link
- **Graph** is a mathematical representation of a network
 - Web graph, Social graph (a Facebook term)
- Language: Graph, vertex, edge
- We will try to make this distinction whenever it is appropriate, but in most cases we will use the two terms interchangeably

Networks: Common Language



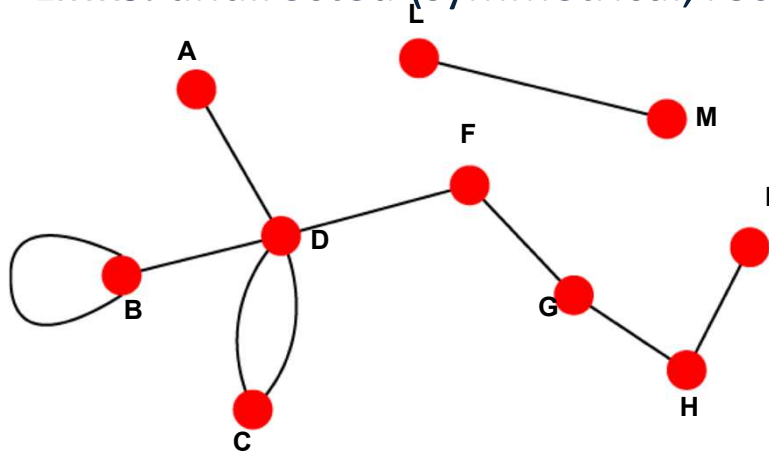
How do you define a graph?

- How to build a graph:
 - What are nodes?
 - What are edges?
- **Choice of the proper graph representation** of a given domain/problem determines our ability to use networks successfully:
 - In some cases there is a unique, unambiguous representation
 - In other cases, the representation is by no means unique
 - The way you assign links will determine the nature of the question you can study

Undirected vs. Directed Graphs

Undirected

Links: undirected (symmetrical, reciprocal)

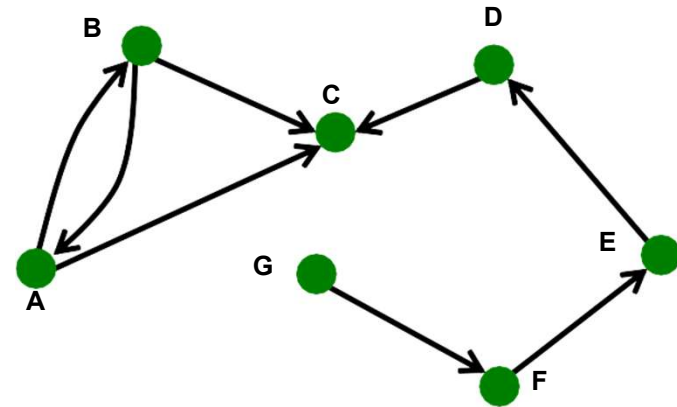


Examples:

- Collaborations
- Friendship on Facebook

Directed

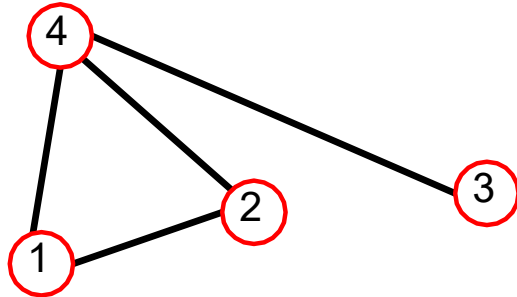
Links: directed (arcs)



Examples:

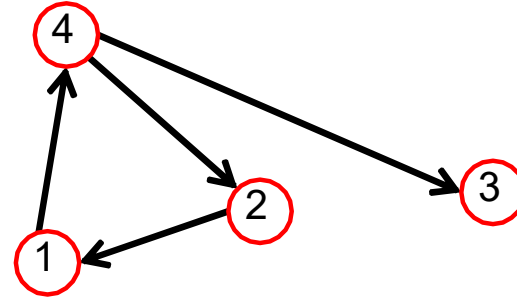
- Phone calls
- Following on Twitter

Adjacency Matrix



$A_{ij} = 1$ if there is a link from node i to node j
 $A_{ij} = 0$ otherwise

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

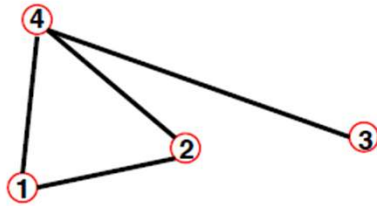


$$A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Note that for a directed graph (right) the matrix is not symmetric.

Adjacency Matrix

Undirected



$$A_{ij} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

$$A_{ij} = A_{ji}$$

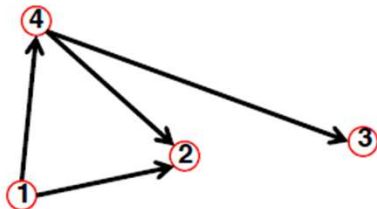
$$A_{ii} = 0$$

$$k_i = \sum_{j=1}^N A_{ij}$$

$$k_j = \sum_{i=1}^N A_{ij}$$

$$L = \frac{1}{2} \sum_{i=1}^N k_i = \frac{1}{2} \sum_{ij} A_{ij}$$

Directed



$$A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

$$A_{ij} \neq A_{ji}$$

$$A_{ii} = 0$$

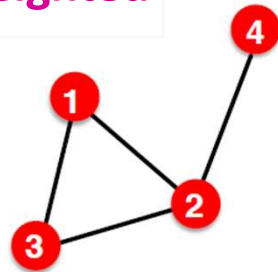
$$k_i^{out} = \sum_{j=1}^N A_{ij}$$

$$k_j^{in} = \sum_{i=1}^N A_{ij}$$

$$L = \sum_{i=1}^N k_i^{in} = \sum_{j=1}^N k_j^{out} = \sum_{i,j} A_{ij}$$

Unweighted vs. Weighted Graphs

Unweighted



$$A_{ij} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

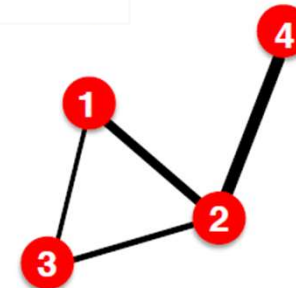
$$A_{ii} = 0 \quad A_{ij} = A_{ji}$$

$$E = \frac{1}{2} \sum_{i,j=1}^N A_{ij} \quad \bar{k} = \frac{2E}{N}$$

Examples:

- Friendship, Hyperlink

Weighted



$$A_{ij} = \begin{pmatrix} 0 & 2 & 0.5 & 0 \\ 2 & 0 & 1 & 4 \\ 0.5 & 1 & 0 & 0 \\ 0 & 4 & 0 & 0 \end{pmatrix}$$

$$A_{ii} = 0 \quad A_{ij} = A_{ji}$$

$$E = \frac{1}{2} \sum_{i,j=1}^N \text{nonzero}(A_{ij}) \quad \bar{k} = \frac{2E}{N}$$

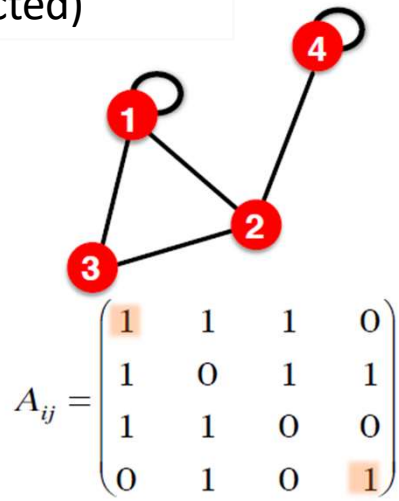
Examples:

- Collaboration, Internet, Roads

Self-edges (self-loops) Graphs vs. Multigraph

Self-edges (self-loops)

(undirected)



$$A_{ii} \neq 0 \quad A_{ij} = A_{ji}$$

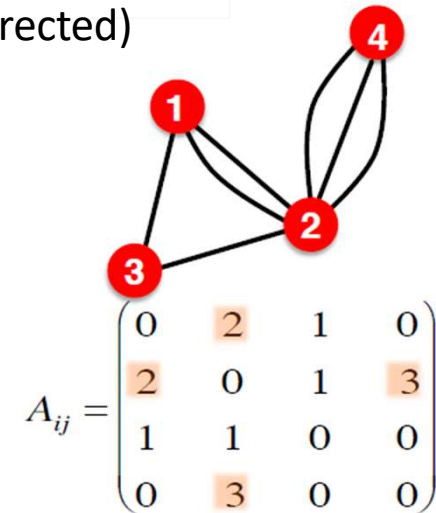
$$E = \frac{1}{2} \sum_{i,j=1, i \neq j}^N A_{ij} + \sum_{i=1}^N A_{ii}$$

Examples:

- Proteins, Hyperlinks

Multigraph

(undirected)



$$A_{ii} = 0 \quad A_{ij} = A_{ji}$$

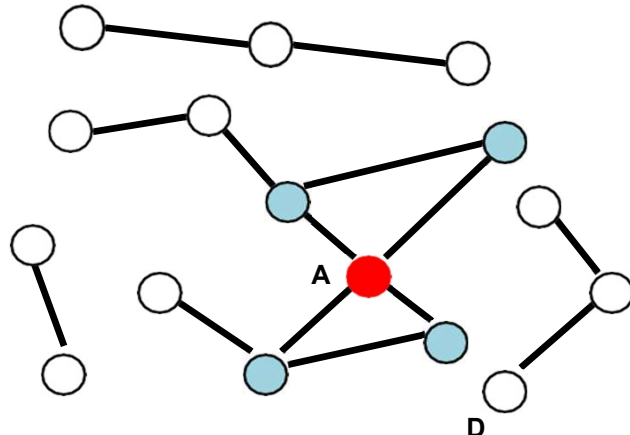
$$E = \frac{1}{2} \sum_{i,j=1}^N \text{nonzero}(A_{ij}) \quad \bar{k} = \frac{2E}{N}$$

Examples:

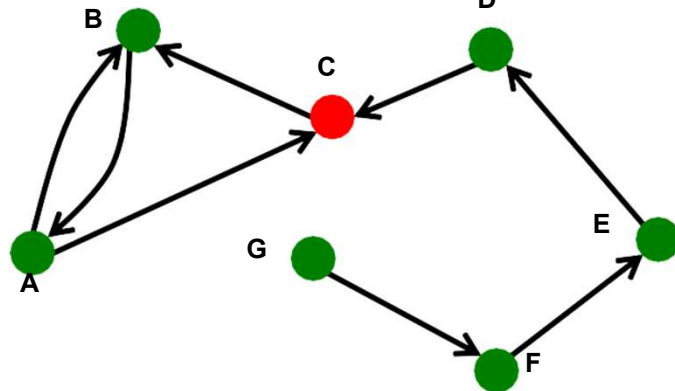
- Collaboration, Communication

Node Degrees

Undirected



Directed



Source: Node with $k^{in} = 0$

Sink: Node with $k^{out} = 0$

Node degree, k_i : the number of edges adjacent to node i

$$k_A = 4$$

Avg. degree: $\bar{k} = \langle k \rangle = \frac{1}{N} \sum_{i=1}^N k_i = \frac{2E}{N}$

In directed networks we define an **in-degree** and **out-degree**.

The (total) degree of a node is the sum of in- and out-degrees.

$$k_C^{in} = 2 \quad k_C^{out} = 1 \quad k_C = 3$$

$$\bar{k} = \frac{E}{N}$$

$$\bar{k}^{in} = \bar{k}^{out}$$

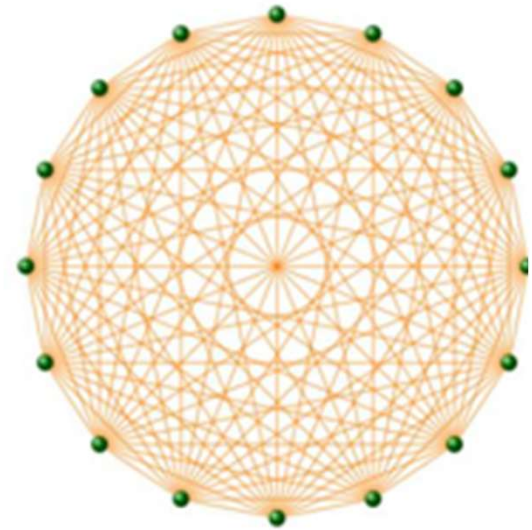


UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Complete Graph

The **maximum number of edges** in an undirected graph on N nodes is

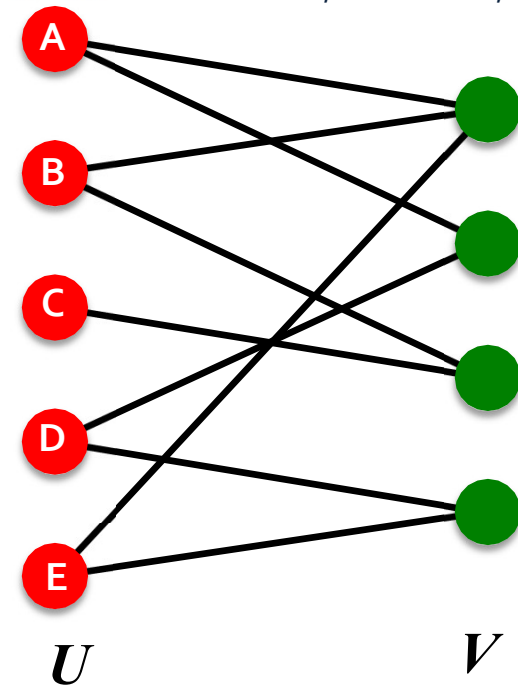
$$E_{\max} = \binom{N}{2} = \frac{N(N-1)}{2}$$



An undirected graph with the number of edges $E = E_{\max}$ is called a **complete graph**, and its average degree is $N - 1$

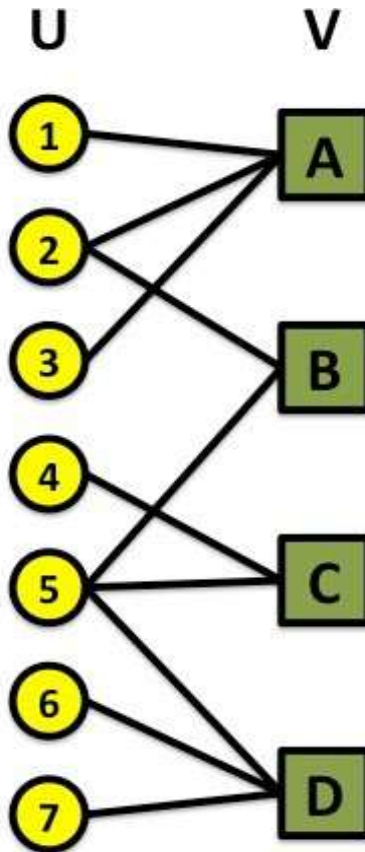
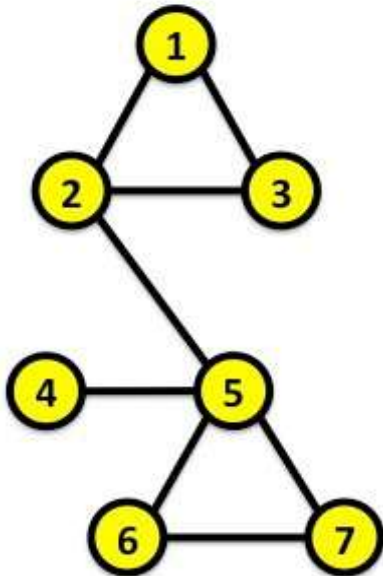
Bipartite Graph

- **Bipartite graph** is a graph whose nodes can be divided into two disjoint sets **U** and **V** such that every link connects a node in **U** to one in **V**; that is, **U** and **V** are **independent sets**
- Examples:
 - Authors-to-Papers (they authored)
 - Actors-to-Movies (they appeared in)
 - Users-to-Movies (they rated)
 - Recipes-to-Ingredients (they contain)
- “Folded” networks:
 - Author collaboration networks
 - Movie co-rating networks

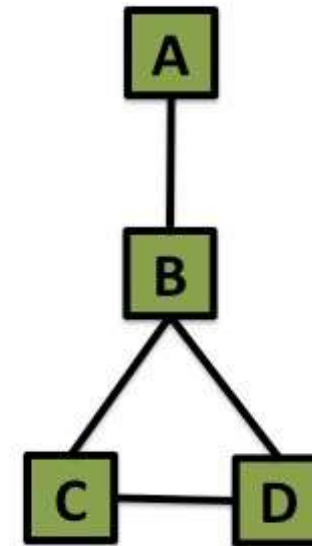


Folded/Projected Bipartite Graphs

Projection U

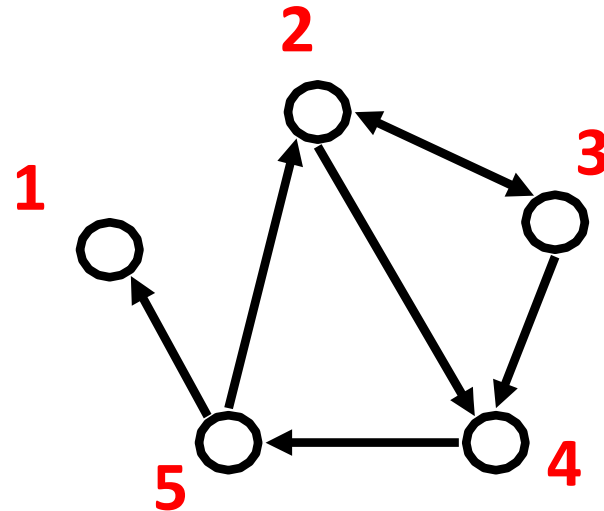


Projection V



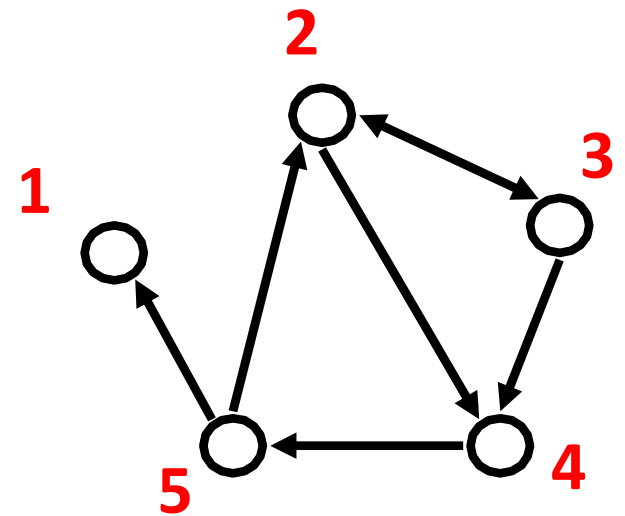
Representing Graphs: Edge List

- Represent graph as a set of edges:
 - (2, 3)
 - (2, 4)
 - (3, 2)
 - (3, 4)
 - (4, 5)
 - (5, 2)
 - (5, 1)



Representing Graphs: Adjacency List

- Adjacency list:
 - Easier to work with if network is
 - Large
 - Sparse
- Allows us to quickly retrieve all neighbors of a given node
 - 1:
 - 2: 3, 4
 - 3: 2, 4
 - 4: 5
 - 5: 1, 2

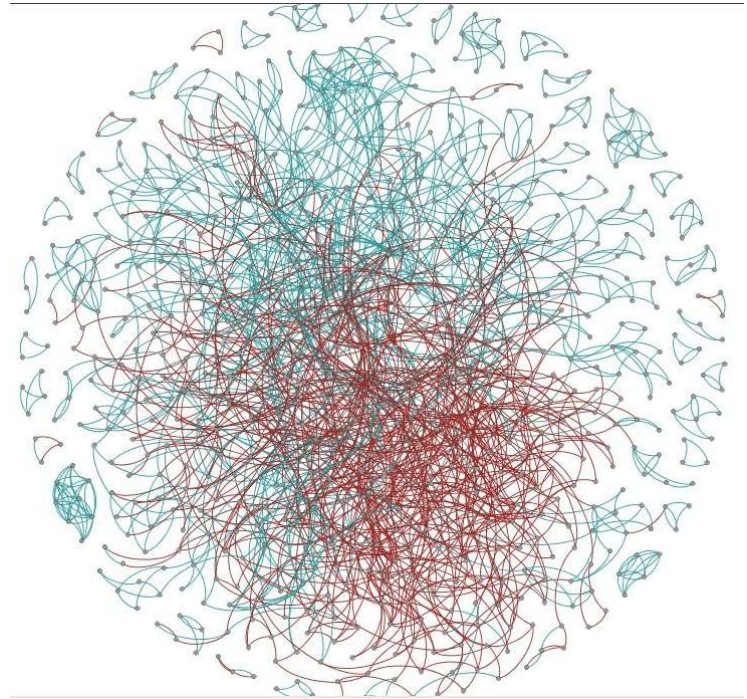


Representing Graphs: Edge Attributes

- Possible options
 - Weight (e.g. frequency of communication)
 - Ranking (best friend, second best friend...)
 - Type (friend, relative, co-worker)
 - Sign: Friend vs. Foe, Trust vs. Distrust
 - Properties depending on the structure of the rest of the graph:
number of common friends

Representing Graphs: Edge Weight

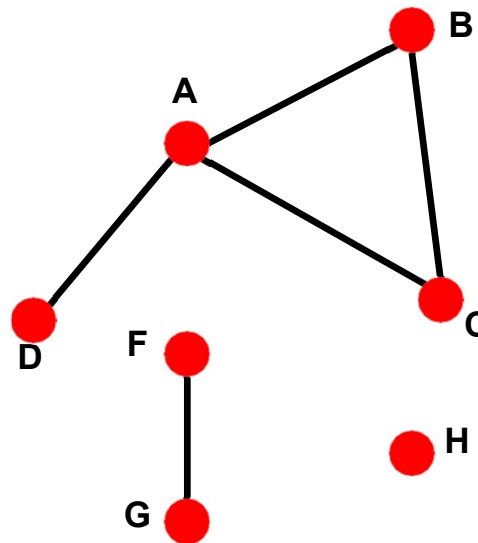
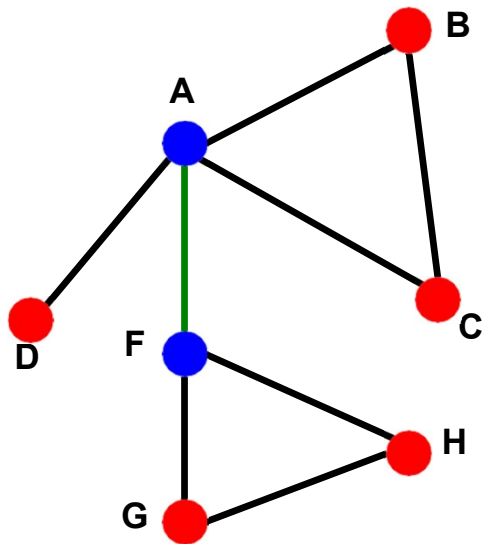
- One person trusting/distrusting another
 - Research challenge
 - How does one 'propagate' negative feelings in a social network?
 - Is my enemy's enemy my friend?



sample of positive & negative ratings from Epinions network

Graph Connectivity of Undirected Graphs

- Connected (undirected) graph: Any two vertices can be joined by a path
- A disconnected graph is made up by two or more connected components



Largest Component:
Giant Component

Isolated node (node H)

Bridge edge: If we erase the **edge**, the graph becomes disconnected

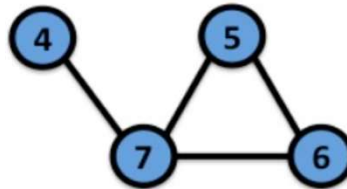
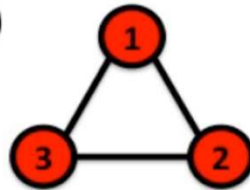
Articulation node: If we erase the **node**, the graph becomes disconnected

Example

- The adjacency matrix of a network with several components can be written in a block-diagonal form, so that nonzero elements are confined to squares, with all other elements being zero:

Disconnected

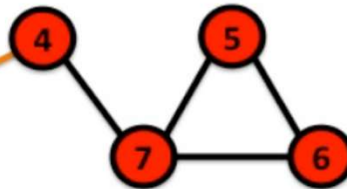
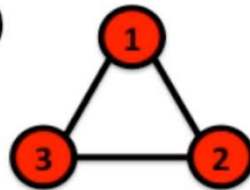
(a)



$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Connected

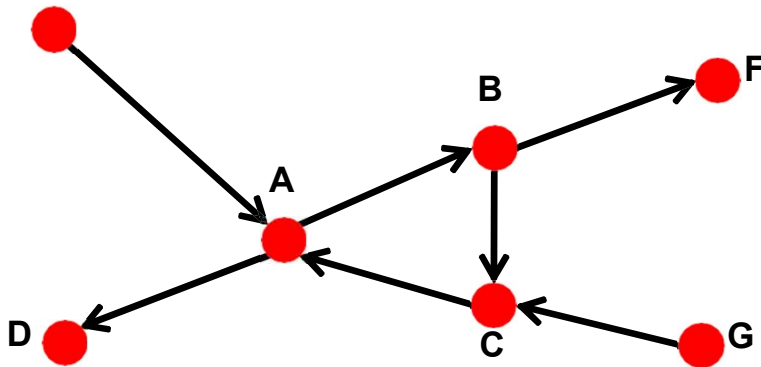
(b)



$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Graph Connectivity of Directed Graphs

- Strongly connected directed graph
 - has a path from each node to every other node and vice versa (e.g., A-B path and B-A path)
- Weakly connected directed graph
 - is connected if we disregard the edge directions



Graph on the left is connected but not strongly connected (e.g., there is no way to get from F to G by following the edge directions).

Network Representations

- Email network >> directed multigraph with self-edges
- Facebook friendships >> undirected, unweighted
- Citation networks >> unweighted, directed, acyclic
- Collaboration networks >> undirected multigraph or weighted graph
- Mobile phone calls >> directed, (weighted?) multigraph
- Protein Interactions >> undirected, unweighted with self-interactions

Network Properties:

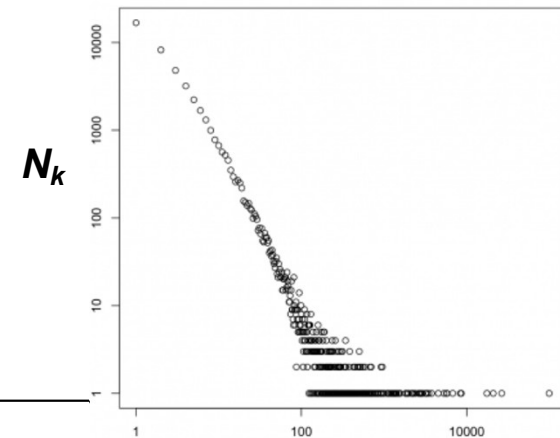
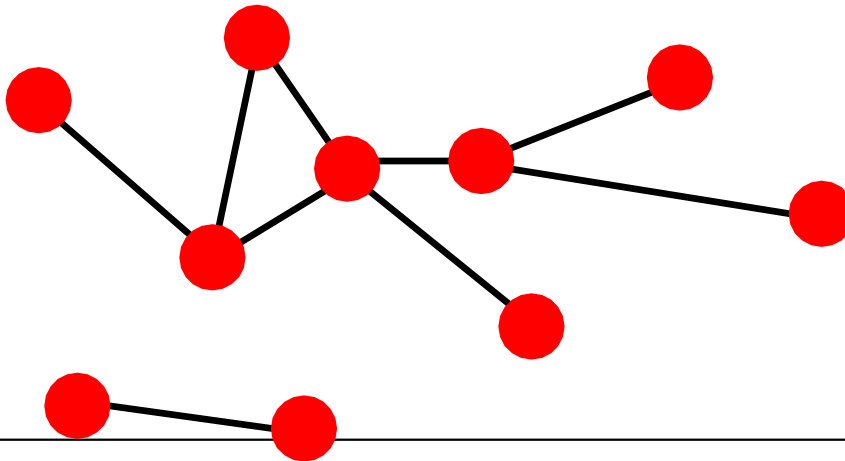
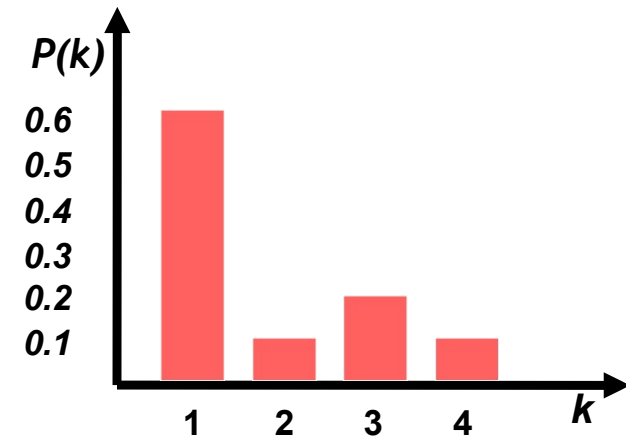
How to Measure a Network

Plan: Key Network Properties

- Degree distribution: $P(k)$
- Path length: h
- Clustering coefficient: C
- Connected components: s

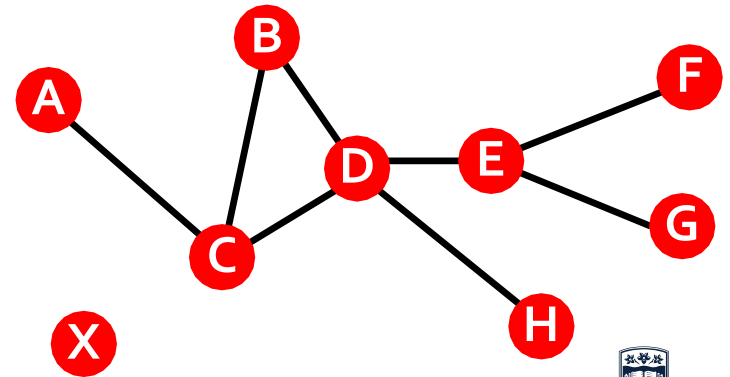
(1) Degree distribution

- Degree distribution $P(k)$: Probability that a randomly chosen node has degree k
 - $N_k = \#$ nodes with degree k
- Normalized histogram:
 - $P(k) = \frac{N_k}{N} \rightarrow$ plot



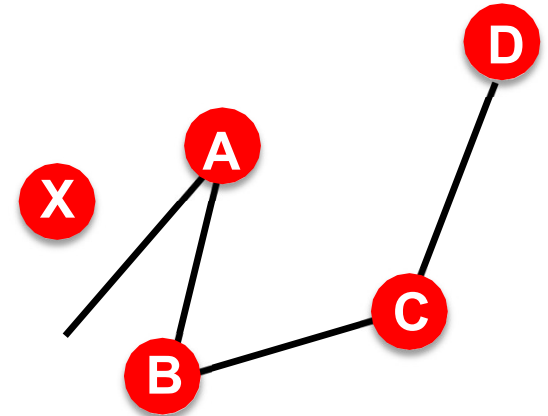
(2) Paths in a Graph

- A path is a sequence of nodes in which each node is linked to the next one
- $P_n = \{i_0, i_1, i_2, \dots, i_n\}$ $P_n = \{(i_0, i_1), (i_1, i_2), (i_2, i_3), \dots, (i_{n-1}, i_n)\}$
- Path can intersect itself and pass through the same edge multiple times
 - E.g.: ACBDCDEG
 - In a directed graph a path can only follow the direction of the “arrow”

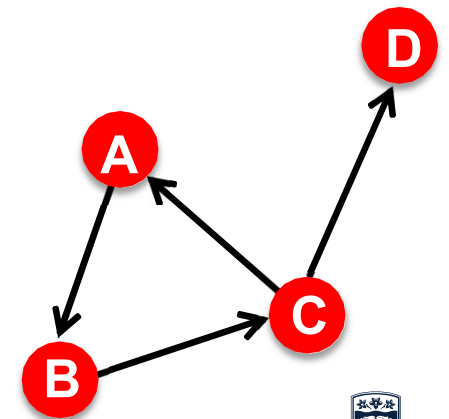


Distance in a Graph

- Distance (shortest path, geodesic) between a pair of nodes is defined as the number of edges along the shortest path connecting the nodes
 - If the two nodes are not connected, the distance is usually defined as infinite
- In directed graphs paths need to follow the direction of the arrows
 - Consequence: Distance is not symmetric:
 $h_{B,C} \neq h_{C,B}$



$$h_{B,D} = 2$$
$$h_{A,X} = \infty$$



$$h_{B,C} = 1, h_{C,B} = 2$$



Network Diameter

- Diameter: The maximum (shortest path) distance between any pair of nodes in a graph
- Average path length for a connected graph (component) or a strongly connected (component of a) directed graph

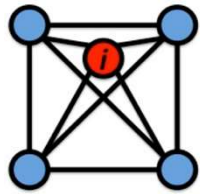
$$\bar{h} = \frac{1}{2E_{\max}} \sum_{i,j \neq i} h_{ij}$$

where h_{ij} is the distance from node i to node j
 $E_{\max}$ is max number of edges (total number of node pairs) = $n(n-1)/2$

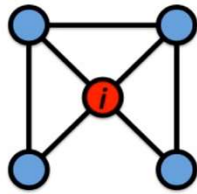
Many times we compute the average only over the connected pairs of nodes (that is, we ignore “infinite” length paths)

Clustering Coefficient

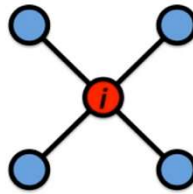
- Clustering coefficient:
 - What portion of i 's neighbors are connected?
 - Node i with degree k_i
 - $C_i \in [0,1]$
 - $C_i = \frac{2e_i}{k_i(k_i-1)}$ where e_i is the number of edges between the neighbors of node i



$$C_i = 1$$



$$C_i = 1/2$$



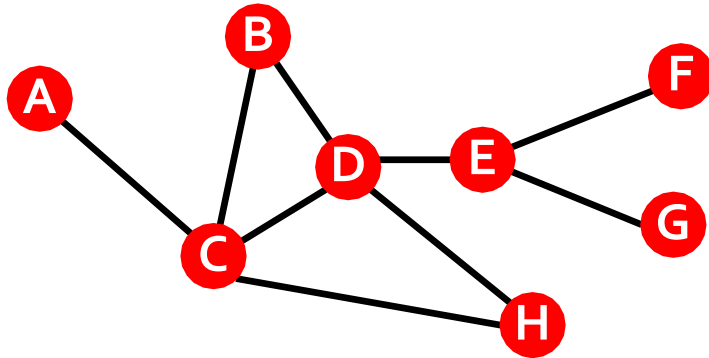
$$C_i = 0$$

Average clustering coefficient:

$$C = \frac{1}{N} \sum_i C_i$$

Clustering Coefficient

- Clustering coefficient:
 - What portion of i 's neighbors are connected?
 - Node i with degree k_i
 - $C_i = \frac{2e_i}{k_i(k_i-1)}$ where e_i is the number of edges between the neighbors of node i



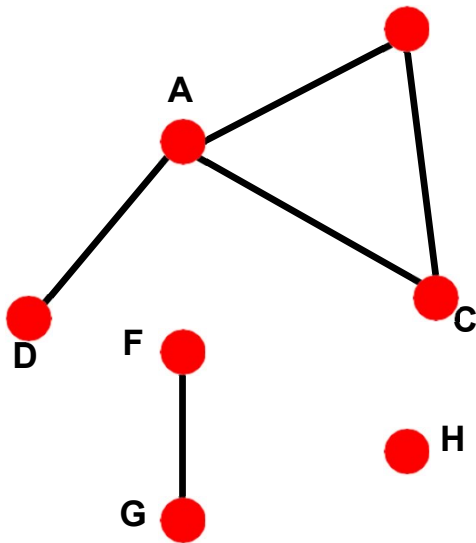
$$k_B=2, \quad e_B=1, \quad C_B=2/2 = 1$$

$$k_D=4, \quad e_D=2, \quad C_D=4/12 = 1/3$$

$$\text{Avg. clustering: } C=0.33$$

(4) Connectivity

- Size of the largest connected component
 - Largest set where any two vertices can be joined by a path
- **Largest component = Giant component**



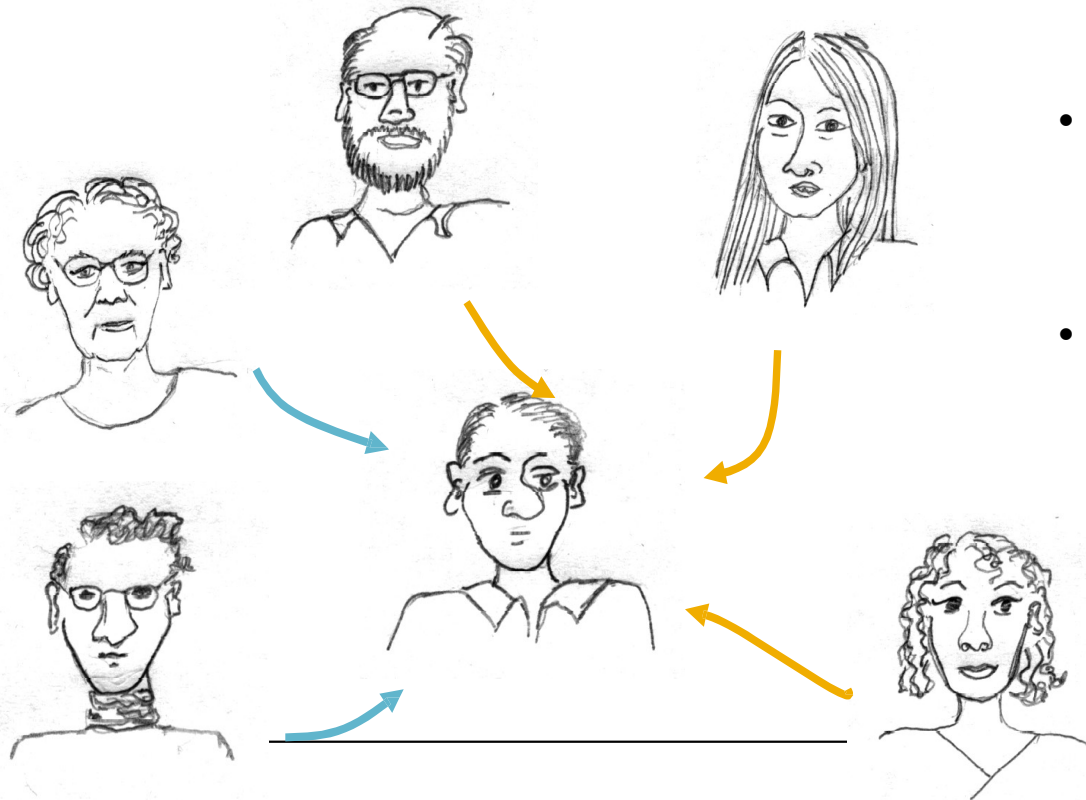
How to find connected components:

- Start from random node and perform Breadth First Search (BFS)
- Label the nodes BFS visited
- If all nodes are visited, the network is connected
- Otherwise find an unvisited node and repeat BFS

An Application: Influence Maximization in Networks

Background: Viral Marketing

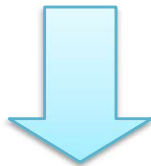
- We are more influenced by our friends than strangers



- 68% of consumers consult friends and family before purchasing home electronics
- 50% do research online before purchasing electronics

Viral Marketing

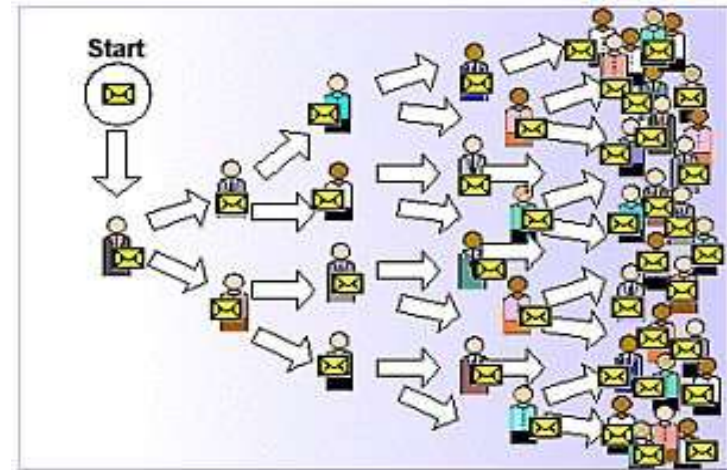
Identify influential customers



Convince them to adopt the product – Offer discount or free samples



These customers endorse the product among their friends



Kate Middleton Effect



“Kate Middleton effect

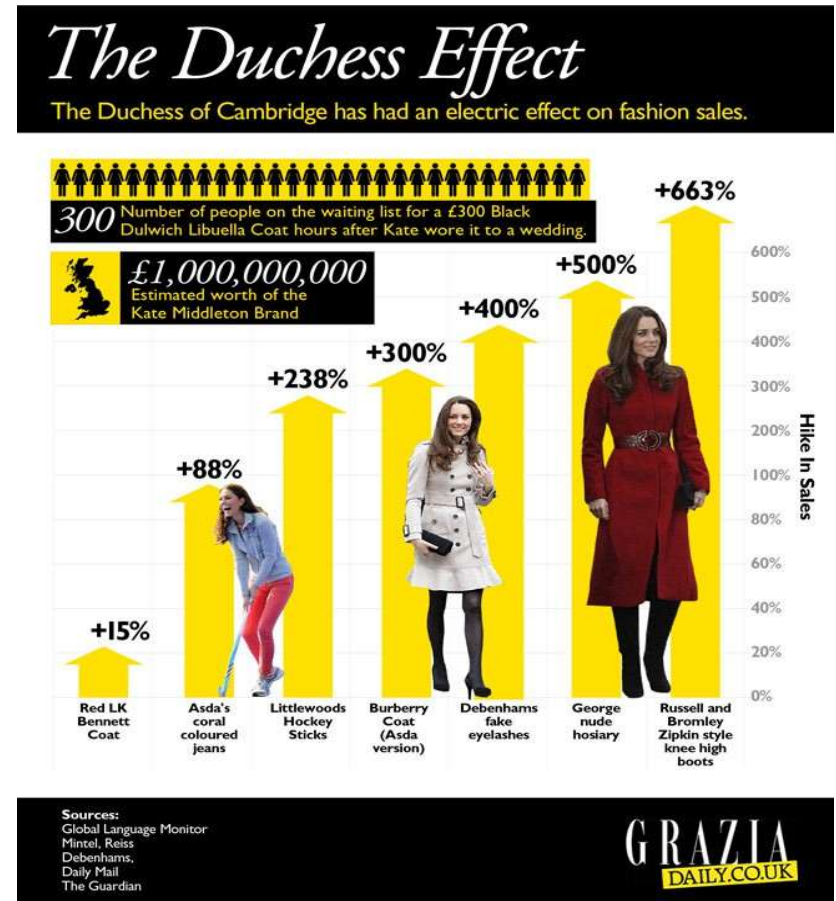
The trend effect that Kate, Duchess of Cambridge has on others, from cosmetic surgery for brides, to sales of coral-colored jeans.”



WIKIPEDIA
The Free Encyclopedia

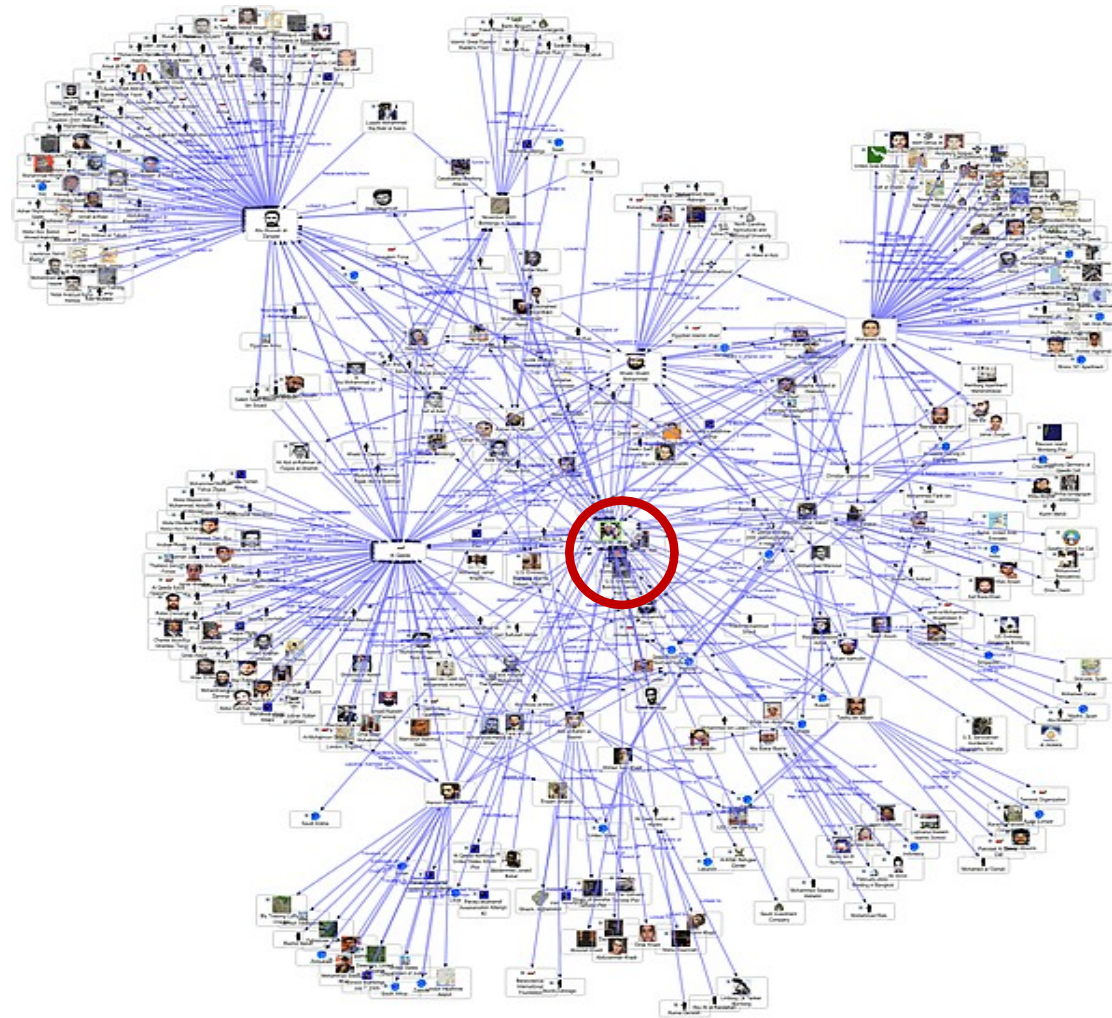
Hike in Sales of Special Products

- According to Newsweek, "The Kate Effect may be worth **£1 billion** to the UK fashion industry."
- Tony DiMasso, L. K. Bennett's US president, stated in 2012, "...when she does wear something, it always seems to go on a **waiting list**."



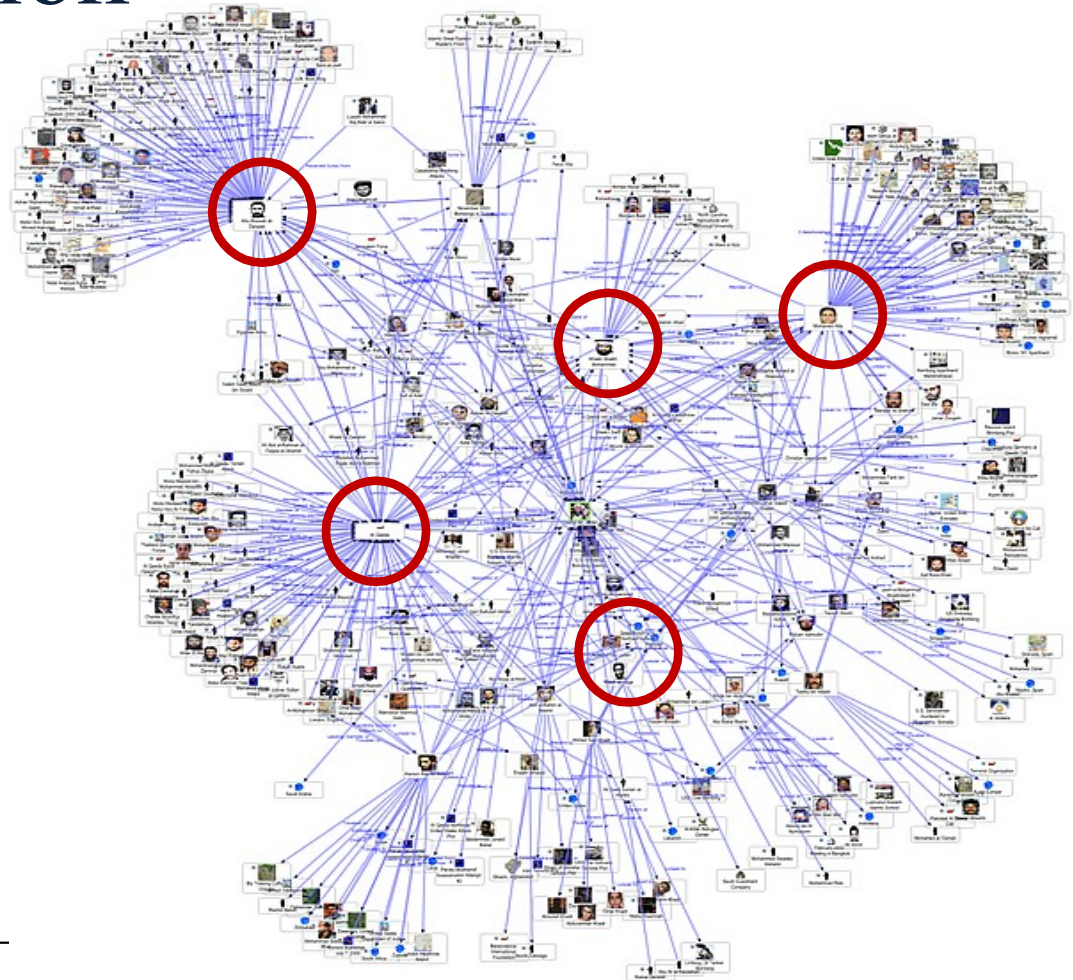
How to Find Kate?

- Influential persons often have many friends
- Kate is one of the persons that have many friends in this social network
- For more Kates, it's not as easy as you might think!



Influence Maximization

- Given a directed graph and $k > 0$, find k seeds (Kates) to maximize the number of influenced people (possibly in many steps)



Two Classical Propagation Models

- Linear Threshold Model
- Independent Cascade Model

Linear Threshold Model

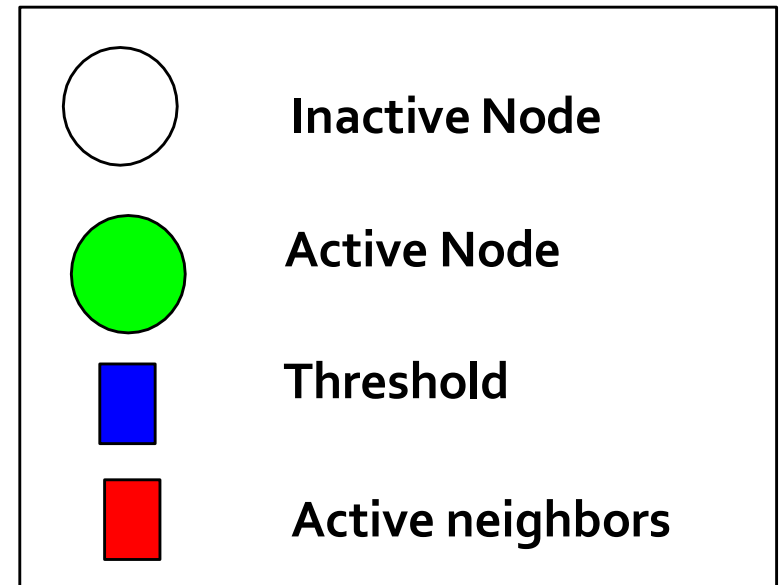
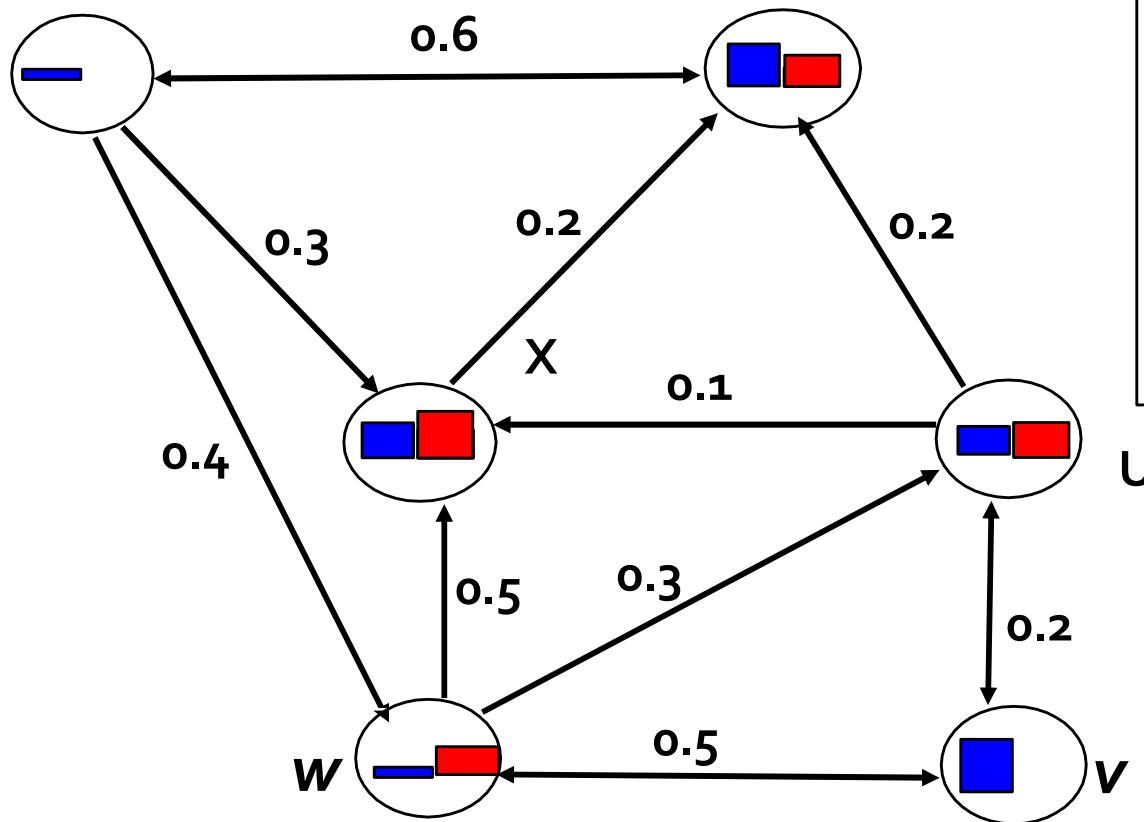
- A node v has random threshold $\theta_v \sim U[0,1]$
- A node v is influenced by each neighbour w according to a weight $b_{v,w}$ such that

$$\sum_{w \text{ neighbor of } v} b_{v,w} \leq 1$$

- A node v becomes active when at least (weighted) θ_v fraction of its neighbours are active

$$\sum_{w \text{ active neighbor of } v} b_{v,w} \geq \theta_v$$

Linear Threshold Model



Stop!

Independent Cascade Model

- **How does the model work?**

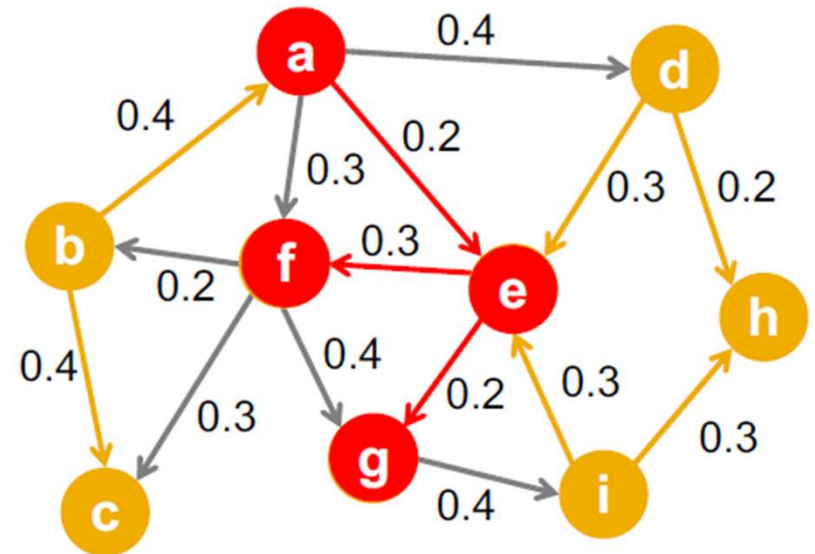
- Directed finite $G = (V, E)$
- Set S starts out with new behavior
 - Say nodes with this behavior are “active”
- Each edge (v, w) has a probability p_{vw}
- If node v is active, it gets **one** chance to make w active, with probability p_{vw}
 - Each edge fires at most once

- **Does scheduling matter? No**

- If u, v are both active at the same time, it doesn't matter which tries to activate * first
- But the time moves in discrete steps

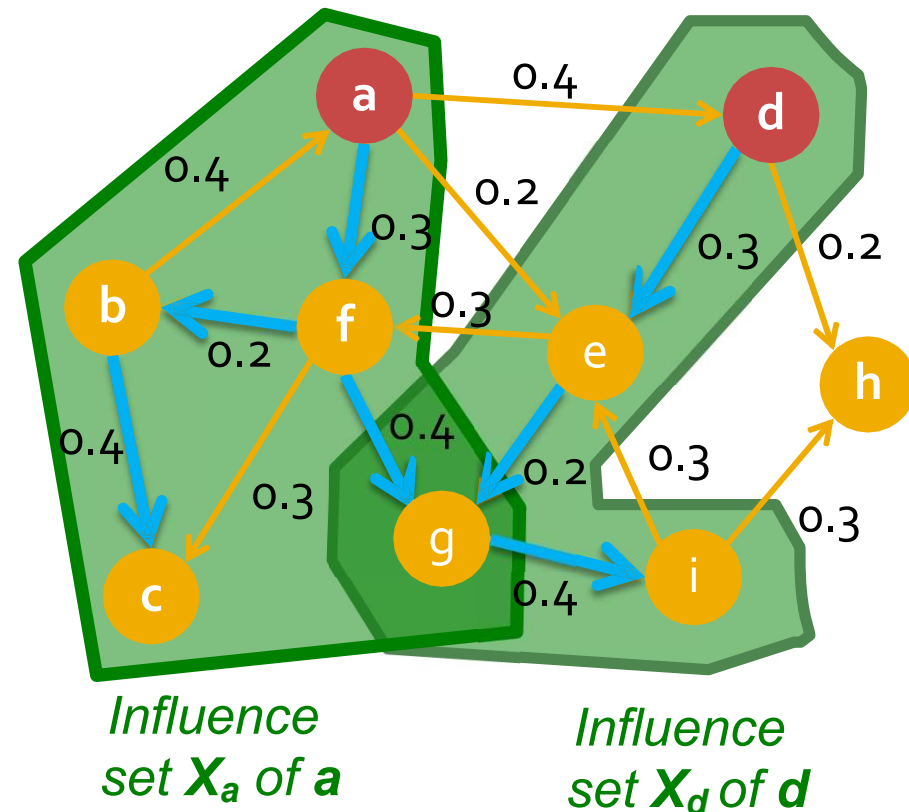
Independent Cascade Model

- Initially some nodes S are active
- Each edge (v, w) has a probability p_{vw}
- When node v becomes active:
 - It activates each out-neighbor w with prob. p_{vw}
- Activations spread through the network**



Most Influential Set

- Problem: (k is a user-specified parameter)
 - Most influential set of size k : set S of k nodes producing **largest expected cascade size $f(S)$** if activated.
- Optimization problem: $\max_{S \text{ of size } k} f(S)$



Why “expected cascade size”? X_a is a result of a random process. So in practice we would want to compute X_a for many random realizations and then maximize the “average” value $f(S)$. For now let’s ignore this nuisance and simply assume that each node u influences a set of nodes X_u .

$$f(S) = \frac{1}{|I|} \sum_{\text{Random realizations } i} f_i(S)$$



U

Thanks

O



W

UNIVERSITY
OF WOLLONGONG
AUSTRALIA